dopingflow

Release 4.1.0

Kazem Zhour

Jul 22, 2026

CONTENTS


dopingflow
AI-Driven Materials Discovery

CHAPTER

ONE

ML-DRIVEN HIGH-THROUGHPUT DOPING WORKFLOW FOR OXIDE MATERIALS

The workflow integrates:

- Structure generation and symmetry-aware enumeration
- ML-based screening and relaxation using configurable backends (M3GNet, UMA, MACE, GRACE)
- Sequential gradual doping with stepwise relaxation and energy recomputation
- Formation energy calculations using configurable thermodynamic reference schemes
- Restricted one-dimensional alloy convex hulls and full multicomponent phase diagrams
- Bandgap prediction using ALIGNN
- Automated database collection
- Fully reproducible, stage-isolated execution

All stages are controlled through a single `input.toml` file.

The workflow is modular: each stage can be executed independently or combined into a full pipeline using the `run-all` command.

CHAPTER

TWO

USER GUIDE

2.1 Workflow Overview

2.1.1 Conceptual Pipeline

The ML Doping Workflow implements a fully automated, multi-stage surrogate pipeline for the exploration of doped crystalline materials.

It combines symmetry-aware structure generation with machine-learned interatomic potentials to efficiently screen large configurational spaces.

The workflow is designed to first identify promising bulk candidates and then optionally extend the analysis to surface structures.

2.1.2 Pipeline Structure

Reference Construction → Enumeration → Screening → Relaxation → Filtering → Band Gap → Formation Energy → Database → 1D Alloy Hull → Phase Diagram → Vacancies

Optional Post-Processing:

Database → Surface Generation → Surface Relaxation

2.1.3 Sequential Workflow

For gradual doping studies, `dopingflow` also provides a sequential workflow:

```
dopingflow sequential-run -c input.toml
```

In this mode, the workflow processes the composition list step by step. For example, a sequence such as 2.5%, 5%, 7.5%, and 10% Sb is treated as a gradual doping path. At each step, the workflow scans possible dopant positions, relaxes the selected candidates, and copies the lowest-energy relaxed structure as the starting point for the next composition.

The sequential workflow supports two modes:

- `full`: run generation, scan, relaxation, filtering, optional bandgap, formation energy, and database collection for each composition.
- `recompute_energies`: reuse existing relaxed sequential structures and recompute formation/mixing energies after changing reference structures.

After `sequential-run` completes, run `dopingflow alloy-hull -c input.toml` to construct the global hull from the merged sequential database.

2.1.4 Stages

0. Reference construction and relaxation
    - Relax host structure (unit cell and supercell)
    - Relax reference phases (metal or oxide mode)
    - Build thermodynamic reference dataset
1. Symmetry-reduced dopant enumeration
    - Generate substitutional doped configurations
    - Identify symmetry-unique arrangements on the cation sublattice
2. ML-based energy screening
    - Evaluate single-point energies using a selected ML backend
    - Supports: M3GNet, UMA, MACE, GRACE
    - Exact enumeration or stochastic sampling
3. Structure relaxation
    - Relax candidate structures using ML forces
    - Uses ASE optimizers (e.g. BFGS, FIRE, LBFGS)
    - CPU or GPU execution
4. Energy-based filtering
    - Select low-energy candidates
    - Window-based or top-N selection strategies
5. Band gap prediction
    - Predict electronic band gaps using ALIGNN
6. Formation energy evaluation
    - Compute formation energies using reference structures
    - Supports metal and oxide reference schemes
7. Database assembly
    - Aggregate results across all stages
    - Export a unified CSV database
8. Restricted one-dimensional alloy hull
    - Construct the lower convex envelope along one fixed substitutional alloy line
    - Compute energy above the restricted 1D hull per cation
    - Identify stable alloy vertices and tie-line decompositions
9. Full phase diagram analysis
    - Build a separate pymatgen hull for each exact candidate chemical system
    - Include compatible reference phases and lower-dimensional candidates
    - Require one elemental terminal reference for every element
    - Write per-system CSV files plus a combined result table

10. Oxygen vacancies (optional run-all extension)
    - Analyze actual selected-parent dopant counts and reachable formal charges
    - Enumerate symmetry-distinct arrangements for every count through the maximum
    - Screen and relax top-k arrangements with one shared ML calculator
    - Keep vacancy results separate from normal thermodynamic databases
11. Surface generation (optional)
    - Select candidates from the database
    - Generate slab structures for chosen Miller indices
    - Enumerate surface terminations
    - Optionally fix atoms in the slab
12. Surface relaxation (optional)
    - Relax slab structures using ML interatomic potentials
    - Apply atom constraints (e.g. fixed bottom layers)
    - Use the same backend abstraction as bulk relaxation

2.1.5 Design Principles

- **Modular**: Each stage can be executed independently
- **Backend-agnostic**: Multiple ML potentials are supported
- **Reproducible**: Fully controlled via `input.toml`
- **Scalable**: Supports multiprocessing and GPU execution
- **Extensible**: New models and stages can be added easily

2.1.6 Notes

- The core workflow (Stages 0–9) focuses on bulk screening, database generation, and thermodynamic analysis.
- Surface generation is intentionally decoupled from the main pipeline and is executed separately.
- This design allows users to: - inspect and validate bulk candidates before surface modeling - control the number of generated slabs - avoid combinatorial explosion of surface structures

2.1.7 Typical Usage

A typical workflow consists of:

1. Running the full bulk pipeline:

```
dopingflow run-all -c input.toml
```

2. Inspecting the resulting database:

```
results_database.csv
```

3. Generating and optionally relaxing surfaces:

```
dopingflow surface -c input.toml
```

2.2 Installation, Usage, and Outputs

This page explains how to install **dopingflow** and how to run the workflow either step-by-step or using the single orchestration command.

2.2.1 Installation

Clone the repository and install in editable mode:

```
git clone KazemZh/dopingflow
cd dopingflow
python -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -e .
```

Verify the CLI is available:

```
dopingflow --help
```

2.2.2 Required Inputs

Refer to *Required Input Files* page.

2.2.3 Running the Workflow

All commands accept `-c/--config` to specify the TOML file. If omitted, `input.toml` in the current directory is used.

Run the full pipeline with one command

To run the complete workflow in order:

```
dopingflow run-all -c input.toml
```

This executes:

```
refs -> generate -> scan -> relax -> filter -> bandgap -> formation -> collect ->
alloy-hull -> phase-diagram
```

Vacancies are an optional final run-all step, leaving the default stop unchanged:

```
dopingflow run-all -c input.toml --until vacancies
dopingflow run-all -c input.toml --only vacancies
```

The surface stage is not included in `run-all` and must be executed separately:

```
dopingflow surface -c input.toml
```

This design allows users to first inspect and validate the final database before generating surface structures.

Resuming and partial runs (run-all)

You can resume from a given stage:

```
dopingflow run-all -c input.toml --from relax
```

You can stop at a stage (inclusive). This is useful if you do not want to run bandgap yet:

```
dopingflow run-all -c input.toml --until filter
```

You can print the planned steps without running them:

```
dopingflow run-all -c input.toml --dry-run
```

You can run only a subset of steps inside a selected range:

```
dopingflow run-all -c input.toml --from refs --until collect --only refs,generate,scan
```

Filtering controls inside run-all

The filter stage supports optional overrides (passed through by `run-all`):

- Restrict filtering to a single composition folder:

  ```
  dopingflow run-all -c input.toml --from relax --until filter --filter-only Sb5_Zr5
  ```

- Force re-filtering even if outputs exist:

  ```
  dopingflow run-all -c input.toml --from filter --until filter --force
  ```

- Override filtering mode by specifying one of:

  ```
  dopingflow run-all -c input.toml --from filter --until filter --window-mev 50
  dopingflow run-all -c input.toml --from filter --until filter --topn 12
  ```

Step-by-step execution

Step 00: build and relax thermodynamic reference structures:

```
dopingflow refs-build -c input.toml
```

Step 01: structure generation:

```
dopingflow generate -c input.toml
```

Step 02: scan (symmetry-unique enumeration / sampling + ML-based single-point energies):

```
dopingflow scan -c input.toml
```

Step 03: relax scanned candidates (ML backend + ASE optimizer):

```
dopingflow relax -c input.toml
```

Step 04: filter relaxed candidates:

```
dopingflow filter -c input.toml
```

Optional Step 05: predict bandgap (ALIGNN):

Before running bandgap, set the model path:

```
export ALIGNN_MODEL_DIR=/path/to/your/alignn/model_root
dopingflow bandgap -c input.toml
```

Step 06: formation energies:

```
dopingflow formation -c input.toml
```

Step 07: collect results into one CSV database:

```
dopingflow collect -c input.toml
```

Step 08: generate surfaces and optionally relax slabs:

```
dopingflow surface -c input.toml
```

Unified oxygen-vacancy workflow:

```
dopingflow vacancies -c input.toml
```

This one command performs charge-count determination, symmetry enumeration, single-point screening, fixed-count top-k selection, and ML relaxation. Its six logged phases are implementation details, not separate public commands.

2.2.4 Outputs Overview

This section summarizes the main outputs created by each stage.

Step 00 (refs-build)

Writes:

- `reference_structures/reference_energies.json`

This file contains:

- relaxed host unit-cell and supercell energies
- relaxed reference structure energies
- metadata about backend, optimizer, device, and convergence settings
- reference information used for formation energy evaluation

Additional outputs:

- `reference_structures/relaxed/host_unit_relaxed.POSCAR`
- `reference_structures/relaxed/host_supercell_<a>x<b>x<c>_relaxed.POSCAR`
- `reference_structures/relaxed/refs/<name>_relaxed.POSCAR`

Step 01 (generate)

Writes a structure folder per composition under `[structure].outdir` (default: `random_structures`):

- `<outdir>/<composition_tag>/POSCAR`
- `<outdir>/<composition_tag>/metadata.json`

Step 02 (scan)

Inside each <composition_tag>/ folder, writes:

- ranking_scan.csv (top-k single-point energies)
- scan_summary.txt (human-readable summary)

Candidate structures:

```
<composition_tag>/candidate_###/01_scan/POSCAR
<composition_tag>/candidate_###/01_scan/meta.json
```

The scan stage evaluates structures using a selected ML backend (e.g. M3GNet, UMA, MACE, GRACE).

Step 03 (relax)

For each candidate:

- candidate_###/02_relax/POSCAR
- candidate_###/02_relax/meta.json

Also writes per composition folder:

- ranking_relax.csv

The relaxation stage uses:

- ML interatomic potentials for forces
- ASE optimizers for structural relaxation

Step 04 (filter)

Writes per composition folder:

- ranking_relax_filtered.csv (filtered candidate table)
- selected_candidates.txt (names of kept candidates)

Step 05 (bandgap)

Writes per composition folder:

- bandgap_alignn_summary.csv

Writes per candidate:

- candidate_###/03_band/meta.json

Step 06 (formation)

Writes per composition folder:

- formation_energies.csv

Writes per candidate:

- candidate_###/04_formation/meta.json

Step 07 (collect)

Writes one flat CSV in the workflow root:

- `results_database.csv`

This file is a compact "database view" across compositions and selected candidates, combining scan/relax/filter/bandgap/formation results where available.

Step 08 (restricted alloy hull)

Writes `alloy_hull_results.csv` and `alloy_hull_summary.json` for the configured substitutional alloy line.

Step 09 (phase diagram)

Writes the combined `phase_diagram_results.csv` plus one exact-system CSV under `phase_diagrams/` for every chemical system represented by candidates.

Step 10 (vacancies, optional)

Each selected parent receives `candidate_###/05_vacancies/` with the parent reference, per-count ranking tables, generated structures, and selected relaxed structures. Global tables remain separate from `results_database.csv`:

```
<structure.outdir>/vacancies_database.csv
<structure.outdir>/vacancies_database.json
```

Optional surface workflow

Generates slab structures from selected candidates and optionally relaxes them.

Input:

- `results_database.csv` (from Step 07)

Writes per candidate:

```
<outdir>/<composition_tag>/candidate_###/hkl_h_k_l/term_###/
```

Files per slab:

- `POSCAR` (generated slab)
- `CONTCAR` (relaxed slab, if enabled)
- `meta.json` (slab metadata)

Optional relaxation outputs:

- `surface_relax.log`
- `surface_relax.traj`
- `surface_relax.json`

Global output:

```
<outdir>/surface_summary.csv
```

The surface stage uses:

- pymatgen for slab generation
- the same ML backend abstraction as Step 03 for relaxation

- ASE optimizers for slab relaxation

2.2.5 Tips

- Use `--verbose` with any command for more detailed logs:

```
dopingflow run-all -c input.toml --verbose
```

- If bandgap is not configured yet (no `ALIGNN_MODEL_DIR`), stop before bandgap:

```
dopingflow run-all -c input.toml --until filter
```

- Run surface generation only after verifying final candidates:

```
dopingflow collect -c input.toml
dopingflow surface -c input.toml
```

- Start with a single composition and one candidate to validate surface settings:

```
composition_tag = "Sb50"
selection_mode = "id"
candidate_id = 1
```

2.2.6 Sequential Doping Workflow

The sequential doping workflow is designed for gradual composition-by-composition doping studies. Instead of generating each target concentration independently from the pristine host, the workflow uses the lowest-energy relaxed structure from the previous composition as the base structure for the next composition.

2.2.7 Command

```
dopingflow sequential-run -c input.toml
```

2.2.8 Execution Logic

For each composition generated from the `[doping]` section, the workflow creates a separate step folder under `[sequential].outdir`:

```
sequential_structures/
   step_001_Sb2p5/
   step_002_Sb5/
   step_003_Sb7p5/
   ...
```

In `mode = "full"`, each step runs:

```
generate -> scan -> relax -> filter -> optional bandgap -> formation -> collect
```

After relaxation, the lowest-energy relaxed candidate is copied to:

```
step_xxx_<composition>/best_relaxed/POSCAR
```

This structure is then used as the base POSCAR for the next sequential step.

2.2.9 Modes

full

Runs the complete sequential workflow for each composition.

```
[sequential]
mode = "full"
```

recompute_energies

Reuses existing relaxed sequential structures and reruns only formation-energy evaluation and database collection.

This is useful when changing the thermodynamic reference, for example:

```
oxides_ref = ["Sb2O5"]
```

without regenerating, scanning, or relaxing structures again.

```
[sequential]
mode = "recompute_energies"
```

2.2.10 Outputs

Each step writes its own local database:

```
sequential_structures/step_001_Sb2p5/results_database.csv
sequential_structures/step_002_Sb5/results_database.csv
```

At the end, all step databases are merged into the project-root database:

```
results_database.csv
```

2.2.11 Bandgap Control

The bandgap stage can be skipped in sequential calculations using:

```
[bandgap]
enabled = false
```

When disabled, the final database is still written, but the `bandgap_eV` column is left empty.

2.3 Required Input Files

This page summarizes **all external files required to run the workflow**.

The workflow itself is code-driven, but it depends on a small number of user-provided structure files and a configuration file.

2.3.1 Overview

At minimum, the following files are required:

1. *input.toml* (workflow configuration)
2. Pristine unit-cell structure of the crystal to be doped (POSCAR format)

Additional reference structure files may be required depending on:

- the selected reference configuration
- whether formation energies are computed
- whether local structure files are used for references

All file paths are interpreted relative to the directory containing `input.toml` unless absolute paths are used.

2.3.2 Directory Layout Example

A clean minimal directory structure may look like:

```
project_root/
    input.toml
    reference_structures/
        base.POSCAR        # pristine unit cell
        host.POSCAR        # host reference structure
        dopant1.POSCAR     # dopant reference
        dopant2.POSCAR     # dopant reference
        dopant3.POSCAR     # dopant reference
        ...
```

This flat layout remains valid as long as the file names and paths match what is specified in `input.toml`.

Structured layouts are also supported, for example:

```
project_root/
    input.toml
    reference_structures/
        oxides/
            SnO2.POSCAR
            Sb2O5.POSCAR
            TiO2.POSCAR
        metals/
            Sn.POSCAR
            Sb.POSCAR
            Ti.POSCAR
        gas/
            O2.POSCAR
```

The exact directory organization is user-defined.

Notes:

- All structure files may be placed inside `reference_structures/` or subdirectories below it.
- `base.POSCAR` is typically the pristine crystal structure used for supercell generation.
- Reference POSCAR files are only required if the corresponding reference mode or formation-energy workflow is used.
- The exact filenames are user-defined, but must match what is specified in `input.toml`.

2.3.3 Pristine Structure

The pristine crystal structure is required for structure generation.

Typical example:

```
reference_structures/base.POSCAR
```

This structure is used to:

- build the supercell
- generate substitutional doped structures
- provide the structural starting point for later workflow stages

2.3.4 Reference Structures

Reference structure files are used for thermodynamic reference construction and downstream formation-energy evaluation.

Depending on the selected workflow setup, these may include:

- host reference structures
- dopant reference structures
- oxide reference structures
- gas reference structures such as O_2

Examples of valid local reference files include:

```
reference_structures/host.POSCAR
reference_structures/dopant1.POSCAR
reference_structures/dopant2.POSCAR
```

or, in a more explicit chemistry-based naming style:

```
reference_structures/metals/Sn.POSCAR
reference_structures/metals/Sb.POSCAR
reference_structures/oxides/Sb2O5.POSCAR
reference_structures/gas/O2.POSCAR
```

These files are interpreted according to the settings in `input.toml`.

2.3.5 ALIGNN Model Directory (Environment Variable)

For bandgap prediction (Step 05), a trained ALIGNN model must be available.

The path must be set via environment variable:

```
export ALIGNN_MODEL_DIR=/path/to/alignn/model
```

This directory must contain:

- `config.json`
- `checkpoint_*.pt`

Without this variable, Step 05 will fail.

2.3.6 Important Notes

- Dopant unit-cell POSCAR files are **not required for structure generation**. Doping is substitutional and uses the pristine structure only.
- Reference POSCAR files are only needed if: - formation energies are computed using local references, or - the selected reference mode requires them
- The workflow does **not** require separate dopant unit-cell structures for substitution.
- The workflow may use either a flat local file layout or a more structured directory layout. Both are valid as long as the paths in `input.toml` are correct.

2.3.7 Summary

Minimum to start:

- `input.toml`
- Pristine POSCAR

To enable full workflow including formation energies and bandgaps:

- Reference structure files as required by your reference setup
- ALIGNN model directory

2.4 Input File Specification (input.toml)

The workflow is fully controlled through a single TOML configuration file.

Each stage reads only the parameters relevant to it.

2.4.1 Execution Model

Each stage defines both:

- **what it does** (physics / workflow logic)
- **how it runs** (CPU / GPU / parallelization)

There is **no global hardware section**. Instead:

- `[scan]` controls screening
- `[relax]` controls structural relaxation
- `[bandgap]` controls band gap prediction

Each stage independently defines:

- `device` (cpu or cuda)
- `gpu_id` (for GPU execution)

This design gives full flexibility and avoids cross-stage conflicts.

2.4.2 Execution Behavior

Stage	Device	Strategy
scan	CPU	multiprocessing
relax	CPU / GPU	workers or single GPU
bandgap	CPU	multiprocessing
bandgap	GPU	batched inference

This design allows each stage to control its own parallelization strategy while sharing a consistent hardware configuration.

All paths are interpreted relative to the directory containing `input.toml`.

The following sections are supported:

- `[references]`
- `[structure]`
- `[doping]`
- `[sequential]`
- `[generate]`
- `[scan]`
- `[relax]`
- `[filter]`
- `[bandgap]`
- `[formation]`
- `[database]` (optional)
- `[vacancies]` (required only by the vacancy workflow)

Not all sections are required for every stage. Each stage reads only what it needs.

2.4.3 [vacancies]

The complete oxygen-vacancy workflow is configured in exactly one flat section and run with `dopingflow vacancies -c input.toml`. Parallel arrays represent mixed formal oxidation states without creating TOML subsections:

```
[vacancies]
enabled = true
parent_source = "selected_candidates"
include_parent_reference = true
skip_if_done = true
resume = true
count_mode = "all_reachable"
host_species = "Sn"
host_oxidation_state = 4
vacancy_species = "O"
vacancy_compensation_charge = 2
oxidation_state_elements = ["Sb", "Nb"]
```

(continues on next page)

(continued from previous page)

```
oxidation_state_values = [[3, 5], [5]]
extra_vacancies = 0
max_vacancies_cap = 8
symprec = 1.0e-3
angle_tolerance = 5.0
mapping_tolerance = 1.0
enumeration_mode = "auto"
max_exact_raw_configs = 300000
max_exact_unique_configs = 100000
sample_budget = 20000
sample_batch_size = 256
sample_patience = 4000
sample_seed = 42
sample_max_saved = 50000
minimum_vacancy_distance = 0.0
backend = "mace"
model = "medium-mpa-0"
task = ""
device = "cuda"
gpu_id = 0
n_workers = 1
tf_threads = 1
omp_threads = 1
chunksize = 25
topk_per_vacancy_count = 15
energy_normalization = "per_vacancy"
optimizer = "bfgs"
fmax = 0.05
max_steps = 300
relax_mode = "atoms"
cell_filter = "frechet"
```

The same resolved backend/model/task is used for parent energies, defective single points, and every relaxation. See *Oxygen-Vacancy Workflow* for the full algorithm, output layout, resume rules, and interpretation limits.

For a directory that already contains many composition subdirectories, use:

```
parent_source = "directory"
parent_directory = "Mn-Nb-Ta-Ce-Ru-In-Sn-Sb"
```

Each subdirectory is discovered through its own `selected_candidates.txt`. The directory path remains a flat key in `[vacancies]`.

2.4.4 [references]

Step 00 — Reference construction and relaxation.

This stage prepares **all thermodynamic reference structures** and writes:

```
reference_structures/reference_energies.json
```

It performs:

- Relaxation of the host oxide unit cell
- Construction and relaxation of the host supercell
- Relaxation of metal reference phases (metal mode)
- Relaxation of oxide reference phases (oxide mode)
- Relaxation of O_2 gas (oxide mode)
- Storage of all relaxed POSCAR files for reuse

Common Parameters

reference_mode (string)

Choose reference scheme:

- `"metal"`
- `"oxide"`

skip_if_done (boolean)

Skip reconstruction if JSON cache exists.

fmax (float)

Force convergence criterion used in relaxation (eV/Å).

supercell (array of 3 integers)

Supercell used later for doping. The host supercell is constructed and relaxed at this stage.

host (string)

Chemical formula of the host oxide (e.g. `"SnO2"`).

host_dir (string)

Directory containing `<host>.POSCAR`.

Metal Reference Mode

Used when `reference_mode = "metal"`.

metal_ref (array of strings)

List of metal element symbols used as reference phases.

metals_dir (string)

Directory containing `<Element>.POSCAR` files.

Example:

```
reference_structures/metals/Sn.POSCAR
reference_structures/metals/Sb.POSCAR
```

Oxide Reference Mode

Used when `reference_mode = "oxide"`.

oxides_ref (array of strings)

List of dopant oxide formulas (e.g. `"Sb2O5"`).

oxides_dir (string)

Directory containing oxide POSCAR files.

gas_ref (string)

Gas reference formula (typically `"O2"`).

gas_dir (string)

Directory containing gas POSCAR file.

oxygen_mode (string)

Currently supports:

- `"O-rich"`
- `"O-poor"`

muO_shift_ev (float)

Optional chemical potential shift applied to oxygen (eV).

2.4.5 [structure]

Defines workflow I/O only.

outdir (string, default: "random_structures")

Directory where generated composition folders are written.

This directory becomes the root for:

- Step 01 outputs
- Step 02–06 per-composition subfolders

2.4.6 [doping]

Defines substitutional doping behavior.

host_species (string, required)

Element symbol of the host species to be substituted.

mode (string)

Defines doping mode:

- `"explicit"` — user provides exact compositions.
- `"enumerate"` — workflow constructs compositions combinatorially.

Explicit Mode

compositions (array of tables)

List of dictionaries:

```
compositions = [
    { Sb = 5 },
    { Sb = 5, Zr = 5 }
]
```

Percentages are defined relative to host sites.

Enumerate Mode

dopants (array of strings)

Allowed dopant elements.

must_include (array of strings)

Dopants that must appear in each composition.

max_dopants_total (integer)

Maximum number of distinct dopants per structure.

allowed_totals (array of floats)

Allowed total dopant percentages.

levels (array of floats)

Discrete concentration values per dopant.

2.4.7 [sequential]

Controls the gradual sequential doping workflow executed with:

```
dopingflow sequential-run -c input.toml
```

outdir (string, default: “sequential_structures”)

Directory where sequential step folders are written.

mode (string, default: "full")

Controls the sequential execution mode:

- `"full"` — run the full stepwise workflow: generate → scan → relax → filter → optional bandgap → formation → collect.
- `"recompute_energies"` — reuse existing relaxed sequential structures and rerun only formation energy and database collection. This is useful when the user wants to change the thermodynamic reference, for example from `Sb2O3` to `Sb2O5`, without regenerating or relaxing structures.

Example

```
[sequential]
outdir = "sequential_structures"
mode = "full"
```

2.4.8 [generate]

Step 01 — Structure generation.

This step generates one doped structure per composition by substituting host atoms inside the **relaxed host supercell** produced by `refs-build`.

Important

- `refs-build` must be executed first.
- The relaxed host supercell is loaded from:

  `reference_structures/reference_energies.json`

- No supercell is constructed in this step.

Parameters

seed_base (integer)

Base seed used for deterministic random substitution.

Each composition generates a stable hash-based seed.

poscar_order (array of strings, optional)

Defines element ordering in written POSCAR files.

If empty, pymatgen default ordering is used.

Example:

```
poscar_order = ["Zr", "Ti", "Sb", "Sn", "O"]
```

clean_outdir (boolean, default: true)

If true, existing output directory is deleted before writing new structures.

Output

For each composition:

```
<outdir>/<composition_tag>/
    POSCAR
    metadata.json
```

2.4.9 [scan]

Step 02 — Dopant configuration prescreening using machine-learning interatomic potentials.

For each generated structure folder inside `[structure].outdir`:

1. Generates doped configurations on the cation sublattice
2. Identifies symmetry-unique configurations
3. Evaluates single-point energies using the selected ML backend
4. Ranks configurations by energy
5. Keeps the lowest-energy `topk` candidates
6. Writes candidate folders and ranking files

Depending on the scan mode, configurations are obtained either by:

- exact symmetry-unique enumeration
- random symmetry-unique sampling

This step operates only on subfolders created in Step 01.

Parameters

backend (string, default: "m3gnet")

Selects the ML model used for energy prediction.

Available options:

- `"m3gnet"` — TensorFlow-based universal interatomic potential
- `"uma"` — FAIR-Chem universal model (requires Hugging Face access)
- `"mace"` — MACE foundation models (MP, MPA, OMAT, MATPES, and MH)
- `"grace"` — GRACE graph neural network models (if installed)

Each backend has its own supported models and execution characteristics.

model (string, default: "default")

Specifies the pretrained model variant used by the selected backend.

Behavior depends on backend:

- `m3gnet`: - `"default"` → loads the standard M3GNet universal model
- `uma`: - `"uma-s-1p2"` - `"uma-s-1p1"` - `"uma-m-1p1"`
- `mace`:
  - accepts every alias exposed by the installed MACE `mace_mp` calculator

  - common aliases include "small", "medium", "large", "medium-mpa-0", "small-omat-0", "medium-omat-0", "mace-matpes-pbe-0", "mace-matpes-r2scan-0", "mh-0", and "mh-1"
  - a local .model, .pt, or .pth checkpoint path is also accepted
  - named model weights are downloaded and cached by MACE on first use
- grace: - GRACE-1L-OMAT - GRACE-1L-OMAT-M-base - GRACE-1L-OMAT-M - GRACE-1L-OMAT-L-base - GRACE-1L-OMAT-L - GRACE-2L-OMAT - GRACE-2L-OMAT-M-base - GRACE-2L-OMAT-M - GRACE-2L-OMAT-L-base - GRACE-2L-OMAT-L - GRACE-1L-OAM - GRACE-1L-OAM-M - GRACE-1L-OAM-L - GRACE-2L-OAM - GRACE-2L-OAM-M - GRACE-2L-OAM-L - GRACE-1L-SMAX-L - GRACE-1L-SMAX-OMAT-L - GRACE-2L-SMAX-M - GRACE-2L-SMAX-L - GRACE-2L-SMAX-OMAT-M - GRACE-2L-SMAX-OMAT-L

task (string, default: "")

Optional task specification (used only for certain backends).

- uma requires a task: - "omat" - "oc20" - "oc22" - "oc25" - "omol" - "odac" - "omc"
- mace optionally uses task as the multi-head model head. For model = "mh-1", leave it empty for MACE's default omat_pbe head or set a supported head explicitly, for example task = "omat_pbe".
- m3gnet, grace: - Not used → keep empty ("")

poscar_in (string, default: "POSCAR")

Filename inside each composition folder used as input.

topk (integer)

Number of lowest-energy configurations retained.

symprec (float)

Tolerance used for symmetry detection in SpacegroupAnalyzer.

mode (string, default: "auto")

Scan strategy.

Possible values:

- auto Automatically chooses exact enumeration for manageable problems and switches to sampling when the configuration space becomes too large.
- exact Forces full symmetry-unique enumeration of all configurations.
- sample Uses random symmetry-unique sampling instead of full enumeration.

max_enum (integer)

Maximum allowed number of raw combinatorial configurations in exact mode.

If this limit is exceeded and mode = "auto", the scan automatically switches to sampling mode.

max_unique (integer)

Maximum allowed number of symmetry-unique configurations in exact mode.

Prevents excessive memory usage for very large configuration spaces.

device (string, default: “cpu”)

Execution device:

- `"cpu"`
- `"cuda"`

Behavior depends on backend:

- `m3gnet`: - GPU mode uses a single worker (TensorFlow limitation)
- `uma`: - GPU supported via PyTorch
- `mace`: - GPU supported and recommended for performance
- `grace`: - GPU support depends on model implementation

n_workers (integer)

Number of parallel worker processes.

- Used only when `device = "cpu"`
- Some backends may internally limit parallelism:
  - `m3gnet` (GPU): forces single worker
  - `mace`: typically runs efficiently in single-process mode
- Ignored when `device = "cuda"` (GPU mode uses a single worker)

chunksize (integer)

Chunk size used in the multiprocessing pool.

Relevant only when `device = "cpu"`.

gpu_id (integer, default: 0)

GPU index used when `device = "cuda"`.

anion_species (array of strings)

Species excluded from substitutional enumeration. Typically contains oxygen:

```
anion_species = ["O"]
```

host_species (from [doping])

Used to define the cation sublattice. Must match the host element defined in `[doping]`.

skip_if_done (boolean, default: true)

If true, skip a structure folder if `ranking_scan.csv` already exists.

Sampling parameters

Used when:

- `mode = "sample"`
- or `mode = "auto"` selects sampling for large configuration spaces

sample_budget (integer)

Maximum number of random sampling attempts.

sample_batch_size (integer)

Number of new symmetry-unique sampled configurations evaluated per batch.

sample_patience (integer)

Sampling stops after this many sampled configurations fail to improve the current best candidate.

sample_seed (integer)

Random seed used for reproducible sampling.

sample_max_saved (integer)

Maximum number of sampled canonical configurations stored to avoid duplicates.

Notes

- If `device = "cuda"`, scan runs on a single GPU and `n_workers` is ignored.
- If `device = "cpu"`, parallelization is controlled via `n_workers` and `chunksize`.
- GPU mode is recommended for faster single-structure evaluation, while CPU mode scales better across many configurations.
- The scan backend can be selected via `backend`.
- Different backends have different accuracy/speed trade-offs:
  - `m3gnet`: stable, general-purpose
  - `uma`: high-quality FAIR-Chem models (requires authentication)
  - `mace`: fast and scalable foundation models
  - `grace`: advanced GNN-based models (experimental)
- For large configuration spaces, sampling mode is recommended.
- GPU acceleration is backend-dependent and may not always scale with multiprocessing.

Output

For each composition folder:

```
ranking_scan.csv
scan_summary.txt
candidate_001/01_scan/POSCAR
candidate_001/01_scan/meta.json
...
```

Each candidate folder contains:

- symmetry-unique configuration
- single-point energy (from selected backend)
- dopant site signature

- scan metadata

The following metadata is recorded:

- backend
- model
- task
- energy_sp_eV
- configuration details

2.4.10 [relax]

Step 03 — Structural relaxation using machine-learning interatomic potentials.

This stage relaxes the low-energy candidates selected in Step 02. For each structure folder in `[structure].outdir`, the workflow reads:

```
candidate_*/01_scan/POSCAR
```

and writes relaxed structures and metadata to:

```
candidate_*/02_relax/
```

Relaxation is performed using the selected ML backend together with an ASE optimizer.

Supported backends:

- `"m3gnet"`
- `"uma"`
- `"mace"`
- `"grace"`

Supported optimizers:

- `"bfgs"`
- `"lbfgs"`
- `"fire"`
- `"mdmin"`
- `"quasinewton"`

Relaxation stops when either:

- the maximum atomic force drops below `fmax`
- the number of optimizer steps reaches `max_steps`

Parameters

backend (string, default: “m3gnet”)

Selects the ML backend used for structural relaxation.

Available options:

- "m3gnet" — TensorFlow-based universal interatomic potential
- "uma" — FAIR-Chem universal model (requires Hugging Face access)
- "mace" — MACE foundation models
- "grace" — GRACE graph neural network models (if installed)

model (string, default: "default")

Specifies the pretrained model variant used by the selected relaxation backend.

Behavior depends on backend:

- m3gnet: - "default" → loads the standard pretrained M3GNet model
- uma: - "uma-s-1p2" - "uma-s-1p1" - "uma-m-1p1"
- mace:
  - accepts every alias exposed by the installed MACE mace_mp calculator
  - common aliases include "small", "medium", "large", "medium-mpa-0", "small-omat-0", "medium-omat-0", "mace-matpes-pbe-0", "mace-matpes-r2scan-0", "mh-0", and "mh-1"
  - local .model, .pt, and .pth checkpoint paths are accepted
- grace: - GRACE-1L-OMAT - GRACE-1L-OMAT-M-base - GRACE-1L-OMAT-M - GRACE-1L-OMAT-L-base - GRACE-1L-OMAT-L - GRACE-2L-OMAT - GRACE-2L-OMAT-M-base - GRACE-2L-OMAT-M - GRACE-2L-OMAT-L-base - GRACE-2L-OMAT-L - GRACE-1L-OAM - GRACE-1L-OAM-M - GRACE-1L-OAM-L - GRACE-2L-OAM - GRACE-2L-OAM-M - GRACE-2L-OAM-L - GRACE-1L-SMAX-L - GRACE-1L-SMAX-OMAT-L - GRACE-2L-SMAX-M - GRACE-2L-SMAX-L - GRACE-2L-SMAX-OMAT-M - GRACE-2L-SMAX-OMAT-L

task (string, default: "")

Optional task specification used for uma and multi-head MACE models.

Allowed values for uma:

- "omat"
- "oc20"
- "oc22"
- "oc25"
- "omol"
- "odac"
- "omc"

For MACE, task is passed as the optional model head. Leave it empty for the model default; for example, MACE-MH-1 defaults to omat_pbe. For m3gnet and grace, this parameter is ignored.

relax_mode (string, default: "atoms")

Controls which degrees of freedom are optimized.

Available options:

- "atoms" — relax atomic positions only.
- "full" — relax atomic positions, cell shape, and volume.
- "isotropic" — relax atomic positions with uniform cell scaling.

- `"volume"` — relax volume only while keeping fractional atomic positions fixed.
- `"shape"` — relax cell shape at nearly constant volume.
- `"xy"` — relax only the in-plane `a` and `b` cell lengths, keeping `c` and angles fixed.
- `"cell_only"` — relax the cell while keeping fractional atomic positions fixed.

cell_filter (string, default: "frechet")

ASE cell filter used when `relax_mode` involves cell relaxation.

Available options:

- `"frechet"` — recommended default.
- `"unit"` — simpler direct cell filter.
- `"exp"` — older exponential cell filter, mainly for compatibility.

optimizer (string, default: "bfgs")

ASE optimizer used during relaxation.

Available options:

- `"bfgs"`
- `"lbfgs"`
- `"fire"`
- `"mdmin"`
- `"quasinewton"`

fmax (float)

Maximum force convergence criterion (eV/Å).

Relaxation is considered converged when the maximum atomic force falls below this threshold.

max_steps (integer, default: 300)

Maximum number of optimizer steps.

If this limit is reached before the force threshold is satisfied, the relaxation stops and the final structure is still written to disk.

device (string, default: "cpu")

Execution device:

- `"cpu"`
- `"cuda"`

Behavior depends on backend and runtime environment. GPU execution uses a single effective worker.

gpu_id (integer, default: 0)

GPU index used when `device = "cuda"`.

n_workers (integer)

Number of parallel relaxation workers (one candidate per worker process).

Relevant mainly when `device = "cpu"`.

tf_threads (integer)

TensorFlow thread count per worker.

Mainly relevant for the `m3gnet` backend. Keep small (typically 1) when using multiple workers.

omp_threads (integer)

OpenMP thread count per worker.

Keep small to avoid CPU oversubscription.

skip_if_done (boolean)

Skip an entire composition folder if `ranking_relax.csv` already exists.

skip_candidate_if_done (boolean)

Skip an individual candidate if both of the following already exist:

```
candidate_*/02_relax/meta.json
candidate_*/02_relax/POSCAR
```

Notes

- If `device = "cuda"`, relaxation uses a single effective worker and `n_workers` is ignored.
- If `device = "cpu"`, parallelization is controlled via `n_workers`.
- The relaxed `POSCAR` follows the species ordering defined by `[generate].poscar_order`.
- If `poscar_order` is empty, the default pymatgen ordering is used.

Output

For each candidate:

```
candidate_###/02_relax/POSCAR
candidate_###/02_relax/meta.json
```

For each structure folder:

```
ranking_relax.csv
```

The metadata records:

- backend
- model
- task
- optimizer
- relaxed energy

- convergence target
- final maximum force
- optimizer step count
- convergence status
- walltime
- link to the original scan metadata

2.4.11 [filter]

Step 04 — Candidate selection.

mode (string)

- `"window"`
- `"topn"`

window_meV (float)

Energy window above the lowest relaxed energy (in meV).

Candidates with:

    E_relaxed <= E_min + window_meV

are retained.

A value of 0 keeps only the lowest-energy structure.

If no candidate satisfies the filtering criteria, the workflow raises an error.

max_candidates (integer)

Number of candidates kept when mode = `"topn"`.

skip_if_done (boolean)

Skip filtering if output exists.

2.4.12 [bandgap]

Step 05 — Bandgap prediction using a local ALIGNN model.

Requires environment variable `ALIGNN_MODEL_DIR` pointing to a local ALIGNN model directory.

enabled (boolean, default: true)

If false, `sequential-run` skips the bandgap stage. The final database is still written, but `bandgap_eV` is left empty.

skip_if_done (bool)

If true, previously computed bandgap results are reused.

Behavior: - If `candidate_*/03_band/meta.json` already exists, the stored bandgap value is reused and prediction is skipped for that candidate. - The summary CSV is rebuilt from existing metadata. - This allows safe re-running of the workflow without recomputing already processed candidates.

If a candidate prediction fails: - The error is recorded in `candidate_*/03_band/meta.json`. - The workflow continues with remaining candidates. - Failed candidates appear in the summary CSV with `NaN` bandgap.

cutoff (float)

Neighbor cutoff radius (Å) used to construct the atomic graph for ALIGNN inference. Must be > 0.

max_neighbors (int)

Maximum number of neighbors retained per atom when building the graph. Must be > 0.

device (string, default: "cpu")

Execution device:

- `"cpu"`
- `"cuda"`

gpu_id (integer, default: 0)

GPU index used when `device = "cuda"`.

batch_size (integer, default: 32)

Batch size used for GPU inference.

n_workers (integer)

Number of CPU workers used for parallel bandgap prediction.

Notes

- CPU mode → multiprocessing over structures using `n_workers`
- GPU mode → batched inference using `batch_size`
- `n_workers` is ignored when using GPU

2.4.13 [formation]

Step 06 — Formation energy calculation.

Formation energies are computed using the chemical potentials written by `refs-build` in `reference_structures/reference_energies.json`.

The reference scheme (metal or oxide) is automatically determined from `[references].reference_mode` and no additional user input is required here.

skip_if_done (boolean)

If true, skip formation calculation if `formation_energies.csv` already exists in a composition folder.

normalize (string)

Defines how the reported formation energy is normalized:

- `"total"` — total formation energy (eV)
- `"per_dopant"` — eV per substituted dopant atom
- `"per_host"` — eV per atom in the supercell

relative_enabled (boolean, default: false)

If true, write reference-specific relative formation and mixing-energy columns. In oxide mode these values preserve the already tie-line-referenced per-cation energies and include explicit oxide-endmember provenance.

endpoint_x (string or float, default: “auto”)

Controls endpoint metadata and the compatibility calculation for older result databases. `"auto"` uses the pure oxide endmember for new oxide-reference results. A numeric value must lie in `(0, 1]`.

Formation energies require:

- Successful execution of `refs-build`
- `reference_structures/reference_energies.json`
- Relaxed candidate structures in `candidate_*/02_relax/`

Notes

- The same substitution formula is used for both reference schemes.
- In `metal` mode, elemental metal references define the chemical potentials.
- In `oxide` mode, chemical potentials are derived from oxide references and the chosen oxygen condition (`O-rich` or `O-poor`).
- The selected reference mode is stored in `candidate_*/04_formation/meta.json` for reproducibility.

2.4.14 [database]

Step 07 — Final database collection.

skip_if_done (boolean, default: true)

If true, do not overwrite existing `results_database.csv`.

2.4.15 [phase_diagram]

Step 09 — Per-chemical-system phase-diagram analysis.

skip_if_done (boolean, default: true)

If true and `phase_diagram_results.csv` exists, return the existing result without rebuilding the per-system diagrams.

stable_threshold_eV_per_atom (float, default: 1e-8)

Non-negative energy-above-hull threshold used for the boolean `stable` field. The step writes a combined CSV plus one CSV per exact chemical system under `phase_diagrams/`.

2.4.16 [surface]

Optional surface generation and surface relaxation.

This stage constructs slab models from selected relaxed bulk candidates and optionally relaxes those slabs using machine-learning interatomic potentials.

The stage reads candidate data from a database file:

```
results_database.csv
```

For each selected candidate, the workflow loads the relaxed bulk structure from:

```
candidate_*/02_relax/POSCAR
```

and generates surface slabs for the requested orientations and terminations.

Generated slabs are written to:

```
[surface].outdir / composition_tag / candidate_### / hkl_* / term_*/
```

If enabled, slab relaxation is performed using the same backend abstraction as in Step 03.

Candidate Selection

Selection is performed in two stages:

1. Restrict dataset by composition:
   - `composition_tag`
   - or `composition_tags`
2. Apply `selection_mode` inside the selected composition subset.

Available selection modes:

- `"id"`
- `"ids"`
- `"rank_range"`
- `"top_n"`
- `"filters"`

Orientation Modes

orientation_mode (string)

- `"explicit"` — use Miller indices from `miller_list`
- `"automatic"` — generate indices up to `max_miller`

miller_list (list)

List of Miller indices when using explicit mode.

Example:

```
[[1, 0, 0], [1, 1, 0], [1, 1, 1]]
```

max_miller (integer)

Maximum Miller index used in automatic mode.

max_orientations (integer)

Maximum number of orientations kept in automatic mode.

Slab Construction

min_slab_size (float)

Minimum slab thickness.

min_vacuum_size (float)

Minimum vacuum thickness.

center_slab (boolean)

If true, centers the slab in the simulation cell.

in_unit_planes (boolean)

Controls units of slab and vacuum thickness:

- `true` → values interpreted in crystallographic planes
- `false` → values interpreted in Å (recommended)

lll_reduce (boolean)

Apply LLL lattice reduction.

primitive (boolean)

Build primitive slab if possible.

reorient_lattice (boolean)

Align slab normal with the out-of-plane axis.

orthogonal_c (boolean)

If true, enforces:

- `c perpendicular to (a, b)`

This is strongly recommended for:

- clean POSCAR output
- stable relaxation
- correct slab thickness interpretation

Surface Terminations

termination_mode (string)

- `"all"` — keep all generated terminations
- `"first"` — keep only the first termination

max_terminations_per_orientation (integer)

Maximum number of terminations retained per orientation.

Fixed Atoms

The surface stage can optionally fix atoms in the slab.

Fixed atoms are written in the same `POSCAR` file using selective dynamics:

- fixed → `F F F`
- free → `T T T`

fix_atoms (boolean)

Enable or disable fixed atoms.

fix_region (string)

- `"bottom"` — fix bottom part of slab
- `"middle"` — fix central region

fix_method (string)

- `"layers"` — fix a number of atomic layers
- `"thickness"` — fix a region of given thickness (Å)

fix_n_layers (integer)

Number of layers to fix when using `layers` mode.

fix_thickness_A (float)

Thickness of fixed region (Å) when using `thickness` mode.

fix_layer_tolerance_A (float)

Tolerance used to group atoms into layers along the z-direction.

Notes

- Layer grouping is based on Cartesian z-coordinates.
- Fixed atoms are enforced during relaxation using ASE constraints, not only written to the POSCAR.

Surface Relaxation

relax_surface (boolean)

Enable or disable slab relaxation.

Relaxation is performed using the same backend system as in Step 03.

surface_backend (string, default: "m3gnet")

Available options:

- `"m3gnet"`
- `"uma"`
- `"mace"`
- `"grace"`

surface_model (string, default: "default")

Model selection depends on backend (same behavior as Step 03).

surface_task (string, default: "")

Task/domain for UMA, or an optional MACE model `head`. Leave it empty to use the MACE model default (MACE-MH-1 defaults to `omat_pbe`).

surface_optimizer (string, default: "bfgs")

ASE optimizer:

- `"bfgs"`
- `"lbfgs"`
- `"fire"`
- `"mdmin"`
- `"quasinewton"`

surface_fmax (float)

Maximum force convergence criterion (eV/Å).

surface_max_steps (integer, default: 300)

Maximum number of optimizer steps.

surface_device (string, default: “cpu”)

- `"cpu"`
- `"cuda"`

surface_gpu_id (integer, default: 0)

GPU index used when `device = "cuda"`.

surface_tf_threads (integer)

TensorFlow thread count (for `m3gnet`).

surface_omp_threads (integer)

OpenMP thread count.

Notes

- GPU mode uses a single effective worker.
- CPU mode runs sequentially per slab (no multiprocessing currently).
- Fixed atoms are enforced via ASE `FixAtoms`.

Surface energy

When enabled, the surface stage also evaluates the surface energy of each generated slab.

The surface energy is computed using the standard symmetric slab expression:

$$\gamma = \frac{E_{\mathrm{slab}} - n\,E_{\mathrm{bulk}}}{2A}$$

where:

- E_{slab} is the total energy of the slab (typically from surface relaxation)
- E_{bulk} is the relaxed bulk energy of the parent candidate
- n is the number of bulk-equivalent units contained in the slab
- A is the surface area
- the factor of 2 accounts for the two surfaces of the slab

Requirements

Surface energy is computed only when all of the following conditions are satisfied:

- A slab energy is available (typically when `relax_surface = true`)
- A bulk reference energy is available from the workflow database
- The slab composition is proportional to the parent bulk composition

If any of these conditions are not met, the surface energy is not computed and a status flag is recorded instead.

Output fields

The following fields are written to:

- `meta.json` (per surface)
- `surface_summary.csv` (global summary)

surface_energy_status (string)

Indicates whether surface energy was successfully computed.

Possible values include:

- `"ok"` — surface energy successfully computed
- `"missing_slab_energy"` — slab energy not available
- `"missing_bulk_energy"` — bulk reference energy missing
- `"not_computable_not_proportional"` — slab composition not proportional to bulk
- `"not_computable_missing_species"`
- `"not_computable_extra_species"`
- `"failed: <error>"` — unexpected numerical or runtime failure

surface_energy_eV_A2 (float)

Surface energy expressed in eV/Å^2.

surface_energy_J_m2 (float)

Surface energy expressed in J/m^2.

Conversion used:

$$1\,\mathrm{eV}/^2 = 16.02176634\,\mathrm{J/m}^2$$

surface_energy_reference_bulk_eV (float)

Bulk reference energy used in the calculation.

surface_energy_n_bulk_equiv (float)

Number of bulk-equivalent units contained in the slab.

This value is determined automatically from the ratio of slab and bulk compositions.

Notes

- Surface energies are only well-defined for slabs whose composition is proportional to the parent bulk.
- Non-stoichiometric terminations are not assigned a surface energy in the current implementation.
- Future extensions may include chemical-potential-based surface energies for non-stoichiometric slabs.

Output

For each generated slab:

```
composition_tag/
    candidate_###/
        hkl_h_k_l/
            term_###/
                POSCAR
                CONTCAR
                meta.json
                surface_relax.log
                surface_relax.traj
                surface_relax.json
```

Files:

POSCAR
: Generated slab structure (with optional selective dynamics)

CONTCAR
: Relaxed slab structure (if relaxation is enabled)

meta.json
: Slab metadata

surface_relax.log
: Relaxation log

surface_relax.traj
: ASE trajectory

surface_relax.json
: Relaxation metadata

Global output:

```
surface_summary.csv
```

Notes

- `min_slab_size` and `min_vacuum_size` should be interpreted in Å when `in_unit_planes = false`.
- `orthogonal_c = true` is recommended for correct slab geometry.
- Relaxed structures are saved separately and do not overwrite the initial slab.

2.4.17 Design Principles

- All stages are deterministic given fixed input.
- All randomness is seed-controlled.
- Each stage can be skipped independently.
- Each stage writes metadata for full reproducibility.
- The relaxed host supercell is constructed once and reused.

CHAPTER

THREE

WORKFLOW STAGES

The workflow is organized into modular stages. Each stage can be executed independently and uses its own configuration block.

3.1 0. Reference Energy Construction

3.1.1 Implementation

This stage is implemented in:

```
src/dopingflow/refs.py
```

The public entry point is:

```
run_refs_build(...)
```

3.1.2 Purpose

This stage prepares all thermodynamic reference quantities required for formation energy evaluation of substitutionally doped structures.

The stage performs the following tasks:

- Relax the host oxide unit cell
- Build and relax the host supercell
- Relax reference structures according to the selected reference scheme
- Store all relevant energies and metadata in:

```
reference_structures/reference_energies.json
```

The resulting reference data are later used by the formation-energy stage.

3.1.3 Inputs

This stage uses settings from:

- `[references]`: reference mode, host structure, reference directories, backend settings, relaxation settings, oxygen settings, and caching behavior
- `[doping]`: defines the host species and the dopant set used in later steps

The host supercell is defined in `[references].supercell` and is constructed at this stage.

3.1.4 Execution Model

The references stage uses the same backend abstraction as the main relaxation stage.

For each structure to be optimized, the workflow:

1. Loads the selected ML backend
2. Builds an ASE-compatible calculator
3. Applies the selected ASE optimizer
4. Relaxes the structure until convergence or until `max_steps` is reached

Supported backends include:

- `m3gnet`
- `uma`
- `mace`
- `grace`

Supported optimizers include:

- `bfgs`
- `lbfgs`
- `fire`
- `mdmin`
- `quasinewton`

The execution device is controlled through:

- `device` (`cpu` or `cuda`)
- `gpu_id`

This design keeps the references stage consistent with `relax.py`.

3.1.5 Reference Modes

Two thermodynamic reference schemes are supported.

Metal reference mode

In metal mode, elemental chemical potentials are taken from relaxed elemental reference phases.

For each relevant element i, the workflow relaxes the corresponding metal structure and computes:

$$\mu_i = \frac{E_{\mathrm{metal}}}{N_{\mathrm{atoms}}}$$

where:

- E_{metal} is the relaxed total energy of the elemental reference structure
- N_{atoms} is the number of atoms in that structure

This mode corresponds to equilibrium with elemental reservoirs.

Oxide reference mode

In oxide mode, dopant chemical potentials are derived from oxide reference phases together with the oxygen chemical potential.

For a binary oxide M_xO_y, the chemical potential satisfies:

$$x\mu_M + y\mu_O = E_{M_xO_y}$$

which gives:

$$\mu_M = \frac{E_{M_xO_y} - y\mu_O}{x}$$

The oxygen chemical potential is obtained from the gas reference (typically O_2):

$$\mu_O = \frac{1}{2}E_{O_2} + \Delta\mu_O$$

where:

- E_{O_2} is the relaxed total energy of the oxygen molecule
- $\Delta\mu_O$ is the optional shift defined by `mu0_shift_ev`

The setting `oxygen_mode` is stored for traceability. For example, `O-rich` usually corresponds to:

$$\Delta\mu_O = 0$$

while more oxygen-poor conditions may be represented by a negative shift.

3.1.6 Method Summary

1. Read the host oxide unit-cell structure
2. Relax the host unit cell
3. Build and relax the host supercell
4. Determine the selected reference mode

Metal mode:

   a. Relax elemental metal references

   b. Compute per-atom elemental chemical potentials

Oxide mode:

   a. Relax oxide reference structures

   b. Relax the oxygen gas reference

   c. Compute μ_O

   d. Derive cation chemical potentials from oxide thermodynamics

5. Write all results and metadata to `reference_energies.json`

3.1.7 Formation Energy Framework

The workflow assumes substitutional doping on host sites.

The formation energy is defined as:

$$E_{\mathrm{form}} = E_{\mathrm{doped}} - E_{\mathrm{pristine}} + \sum_i n_i \left(\mu_{\mathrm{host}} - \mu_i\right)$$

where:

- E_{doped} is the relaxed total energy of the doped supercell
- E_{pristine} is the relaxed total energy of the pristine host supercell
- μ_i is the chemical potential of dopant species i
- μ_{host} is the chemical potential of the substituted host species
- n_i is the number of substituted atoms of species i

This corresponds to removing host atoms and inserting dopant atoms while keeping the total lattice size fixed.

The same formal expression is used in both reference modes; only the way the chemical potentials are constructed differs.

3.1.8 Host Reference Energy

The pristine host reference energy is computed by:

1. Reading the host oxide unit cell
2. Relaxing the unit cell
3. Building the requested supercell
4. Relaxing the supercell
5. Extracting the final total energy

The relaxed host supercell is reused by later workflow stages as the starting point for structure generation.

Both atomic positions and lattice vectors are allowed to relax.

3.1.9 Metal Chemical Potentials

For each relevant elemental reference phase, the workflow computes:

$$\mu_i = \frac{E_{\mathrm{metal}}}{N_{\mathrm{atoms}}}$$

These values are used directly in formation-energy evaluation when `reference_mode = "metal"`.

3.1.10 Oxide-Derived Chemical Potentials

When `reference_mode = "oxide"`, the workflow stores relaxed oxide reference energies and the oxygen gas reference energy.

The oxygen chemical potential is computed from the gas reference and the optional oxygen shift. The cation chemical potentials are then derived from the oxide stoichiometry.

For a reduced oxide composition M_xO_y:

$$\mu_M = \frac{E_{M_xO_y}^{(\mathrm{f.u.})} - y\mu_O}{x}$$

where $E_{M_xO_y}^{(\mathrm{f.u.})}$ is the relaxed energy per formula unit of the oxide reference.

3.1.11 Relaxation Method

All reference relaxations use:

- a selected ML interatomic potential backend
- an ASE-compatible calculator
- a user-selected ASE optimizer
- a force-based convergence criterion defined by `fmax`

The maximum number of optimization steps is controlled by `max_steps`.

The backend is selected through:

- `backend`
- `model`
- `task` (required for UMA; optional MACE multi-head model head)

The runtime environment is controlled through:

- `device`
- `gpu_id`
- `tf_threads`
- `omp_threads`

The relaxations are fully unconstrained (cell parameters and atomic positions).

3.1.12 Caching Strategy

If:

```
skip_if_done = true
```

and `reference_energies.json` already exists, this stage is skipped.

This ensures deterministic behavior and avoids unnecessary recomputation.

3.1.13 Outputs

The file `reference_energies.json` contains metadata and energies needed for later stages.

Typical top-level fields include:

- `timestamp`
- `reference_mode`
- `backend`
- `model`
- `task`
- `optimizer`
- `device`
- `gpu_id`
- `host`

- `references`
- `oxide_mode` (only relevant in oxide mode)
- `supercell`
- `config_path`

The `host` block typically contains:

- host formula
- source POSCAR path
- relaxed unit-cell POSCAR path
- relaxed supercell POSCAR path
- number of atoms in unit cell and supercell
- total and per-atom energies
- optimizer step counts
- final force values
- convergence status

The `references` block contains one entry per relaxed reference phase, for example metals, oxides, or gas references.

Each reference entry may include:

- source POSCAR path
- relaxed POSCAR path
- total energy
- per-atom energy
- per-formula-unit energy (when applicable)
- per-molecule energy for O_2 (when applicable)
- optimizer steps
- final force value
- convergence status
- wall time
- backend and optimizer metadata

For oxide mode, the JSON also stores the oxygen reference settings such as:

- `oxides_ref`
- `gas_ref`
- `oxygen_mode`
- `muO_shift_ev`

3.1.14 Notes and Limitations

- This stage does not evaluate doped structures
- Energies are ML-predicted, not DFT total energies
- Reference phase selection strongly affects the resulting chemical potentials
- The workflow currently assumes substitutional doping
- Oxide-mode derivation assumes simple oxide reference chemistry
- No finite-size corrections are applied
- No charge-state corrections are included
- No entropy or temperature effects are considered
- No competing phase stability analysis is performed

3.2 1. Structure Generation

3.2.1 Implementation

This stage is implemented in:

```
src/dopingflow/generate.py
```

The public entry point is:

```
run_generate(...)
```

3.2.2 Purpose

This stage generates an initial set of doped structures starting from a pristine unit cell. The output is a directory of subfolders, each containing a VASP `POSCAR` and a small metadata file describing the effective composition and the random seed used for site selection.

The generation step supports two workflows:

- **Explicit compositions**: the user provides exact dopant percentages.
- **Enumerated compositions**: the workflow constructs a systematic set of dopant combinations and doping levels under user-defined constraints.

3.2.3 Inputs

This stage uses settings from three sections of `input.toml`:

- `[structure]`: provides the pristine structure file and the supercell size.
- `[doping]`: defines the doping mode and composition rules.
- `[generate]`: controls structure-writing details and reproducible randomness.

3.2.4 Method Summary

1. Read the pristine structure from `[structure].base_poscar`.
2. Build the supercell using `[structure].supercell`.
3. Identify all sites matching the substitution host species (`[doping].host_species`).

4. For each requested composition:
    a. Convert requested dopant percentages to integer substitution counts by rounding.
    b. Randomly choose host sites to substitute, using a deterministic seed.
    c. Optionally reorder species in the written POSCAR.
    d. Write `POSCAR` and `metadata.json` to an output subdirectory.

The output directory is `[structure].outdir` (default: `random_structures`).

3.2.5 Composition Handling

Requested vs effective composition

Doping levels are provided as percentages *relative to the number of host sites* in the generated supercell. Since the number of sites is discrete, requested percentages are converted to integer substitution counts by rounding.

The workflow therefore distinguishes:

- **requested composition**: the percentages from the input
- **effective composition**: the percentages implied by the rounded integer counts

If rounding changes any dopant level, warnings are reported and the **effective** composition is stored in the metadata.

A basic consistency check is applied:

- the total number of substituted atoms may not exceed the number of host sites
- the total requested dopant percentage may not exceed 100%

Explicit mode

In explicit mode, each composition is provided directly by the user as a mapping `element -> percent`. Each composition produces exactly one structure.

This mode is recommended when:

- specific compositions are already known or desired
- only a small number of target compositions are needed

Enumerate mode

In enumerate mode, the workflow generates composition dictionaries automatically from:

- a list of possible dopant species
- an optional list of dopants that must appear
- a set of allowed total dopant levels
- a set of discrete per-dopant levels

The workflow enumerates combinations of distinct dopants of size:

$$k \in \{1, \ldots, k_{\max}\}$$

where $k_{\max}$ is controlled by the input parameter `[doping].max_dopants_total`.

Interpretation:

- you may provide many candidate dopant elements in the input

- but each generated structure contains at most `max_dopants_total` distinct dopant species *when using enumerate mode*

(Explicit mode is not inherently limited unless the user adopts the same constraint.)

3.2.6 Reproducible Random Substitution

The actual substitutional sites are selected randomly, but deterministically:

- a **composition tag** is constructed from the effective composition
- a stable seed is derived from the tag and a base seed (`[generate].seed_base`)

This ensures that rerunning the workflow with unchanged input produces identical structures, while still providing randomized site selection.

3.2.7 Directory Naming and Collision Handling

Each generated structure is written to:

```
<outdir>/<composition_tag>/POSCAR
<outdir>/<composition_tag>/metadata.json
```

The folder tag is constructed from the **effective** composition. If two different requested compositions round to the same effective composition, a suffix is added:

```
<tag>__v2, <tag>__v3, ...
```

This guarantees that all generated structures are preserved.

3.2.8 POSCAR Species Ordering

To control the species order in the written POSCAR, you may provide:

```
[generate]
poscar_order = [...]
```

If this list is empty, the structure is written using pymatgen's default ordering. If non-empty, sites are reordered to match the given preference (and any remaining species are appended).

3.2.9 Outputs

For each generated structure:

- `POSCAR`: doped supercell structure
- `metadata.json`: provenance information, including:
  - host species and number of host sites
  - requested and effective compositions
  - rounded substitution counts
  - seed used for deterministic substitution
  - composition tag and input file name

3.2.10 Notes and Limitations

- This stage performs **structure generation only** and does not evaluate stability.
- Rounding is unavoidable for small supercells; larger supercells reduce rounding error.
- Enumerated composition counts can grow combinatorially with the number of dopants and levels; users should choose constraints accordingly.

3.3 Sequential Doping Workflow

The sequential doping workflow is designed for gradual composition-by-composition doping studies. Instead of generating each target concentration independently from the pristine host, the workflow uses the lowest-energy relaxed structure from the previous composition as the base structure for the next composition.

3.3.1 Command

```
dopingflow sequential-run -c input.toml
```

3.3.2 Execution Logic

For each composition generated from the `[doping]` section, the workflow creates a separate step folder under `[sequential].outdir`:

```
sequential_structures/
   step_001_Sb2p5/
   step_002_Sb5/
   step_003_Sb7p5/
   ...
```

In `mode = "full"`, each step runs:

```
generate -> scan -> relax -> filter -> optional bandgap -> formation -> collect
```

After relaxation, the lowest-energy relaxed candidate is copied to:

```
step_xxx_<composition>/best_relaxed/POSCAR
```

This structure is then used as the base POSCAR for the next sequential step.

3.3.3 Modes

full

Runs the complete sequential workflow for each composition.

```
[sequential]
mode = "full"
```

recompute_energies

Reuses existing relaxed sequential structures and reruns only formation-energy evaluation and database collection.

This is useful when changing the thermodynamic reference, for example:

```
oxides_ref = ["Sb2O5"]
```

without regenerating, scanning, or relaxing structures again.

```
[sequential]
mode = "recompute_energies"
```

3.3.4 Outputs

Each step writes its own local database:

```
sequential_structures/step_001_Sb2p5/results_database.csv
sequential_structures/step_002_Sb5/results_database.csv
```

At the end, all step databases are merged into the project-root database:

```
results_database.csv
```

3.3.5 Bandgap Control

The bandgap stage can be skipped in sequential calculations using:

```
[bandgap]
enabled = false
```

When disabled, the final database is still written, but the `bandgap_eV` column is left empty.

3.4 2. Symmetry-Reduced Energy Pre-screening (Scanning)

3.4.1 Implementation

This stage is implemented in:

```
src/dopingflow/scan.py
```

The public entry point is:

```
run_scan(...)
```

3.4.2 Purpose

This stage performs an efficient **energy pre-screening** of dopant arrangements by enumerating **symmetry-unique** configurations on a selected sublattice and evaluating their **single-point energies** using a machine-learned interatomic potential.

For each generated structure folder, the method:

- infers the substitutional sublattice and dopant counts from the input structure
- generates dopant configurations either by **exact enumeration** or **random sampling**
- removes symmetry-equivalent configurations
- predicts a single-point energy for each configuration using the selected machine-learning backend
- keeps the **top-k lowest-energy** candidates for downstream relaxation

3.4.3 Inputs

This stage uses settings from the following sections of `input.toml`:

- `[structure]`: provides the output directory containing generated structures.
- `[doping]`: provides the host species definition.
- `[generate]`: provides the species ordering used when writing POSCAR files.
- `[scan]`: controls enumeration limits, symmetry tolerance, parallelism, and selection.

3.4.4 Method Summary

For each structure subdirectory in `[structure].outdir`:

1. Read the structure file specified by `[scan].poscar_in`.
2. Identify the **enumeration sublattice** (all non-anion sites) and infer: - host species count on the sublattice - dopant species counts on the sublattice
3. Estimate the total number of raw (non-symmetry-reduced) configurations.
4. Construct a parent structure and compute symmetry operations acting on the sublattice.
5. Decide the scan strategy according to `[scan].mode`:
    - `exact`: perform full symmetry-unique enumeration
    - `sample`: generate configurations using random symmetry-unique sampling
    - `auto`: use enumeration when feasible, otherwise switch to sampling
6. Generate configurations:
    - enumeration: iterate over all dopant permutations
    - sampling: randomly generate dopant arrangements and filter duplicates
7. Reduce configurations to symmetry-unique representations.
8. Evaluate single-point energies in parallel using the selected machine-learning backend.
9. Select the lowest-energy `topk` candidates and write them to `candidate_*/01_scan`.
10. Write a CSV ranking file and a human-readable summary.

3.4.5 Backend and Model Selection

The scan stage supports multiple machine-learning backends for energy evaluation.

The backend is selected via `[scan].backend` and controls how energies are computed.

Supported backends include:

- `m3gnet`: - TensorFlow-based universal interatomic potential - robust and general-purpose
- `uma`: - FAIR-Chem universal models (via Hugging Face) - requires authentication and model access permissions
- `mace`:
    - foundation models exposed by the installed MACE `mace_mp` calculator, including MP, MPA, OMAT, MATPES, and multi-head models
    - fast and efficient, especially for large-scale screening
- `grace`: - graph neural network models for atomistic simulations (if installed) - experimental integration

Model selection is controlled via `[scan].model`.

MACE-MH example using the existing `[scan]` parameters:

```
[scan]
backend = "mace"
model = "mh-1"
task = "omat_pbe"
```

For a single-head MACE model such as `small`, keep `task = ""`. A UMA configuration uses the same fields with a required UMA task:

```
[scan]
backend = "uma"
model = "uma-s-1p2"
task  = "omat"
```

Notes:

- `task` is required for UMA. For MACE it optionally selects a multi-head model head; blank uses the model default.
- The available MACE aliases are discovered from the installed `mace-torch` package. A local checkpoint path can be used instead of an alias.
- Models are typically downloaded and cached automatically on first use.

Backend	Framework	GPU	Strength	Notes
m3gnet	TensorFlow	Yes	Stable, general	Slower on CPU
uma	PyTorch	Yes	High accuracy	Needs HF access
mace	PyTorch	Yes	Fast, scalable	Best for scan
grace	PyTorch	Yes	Advanced GNN	Experimental

3.4.6 Enumeration Sublattice Definition

The enumerated sublattice is inferred from the input structure using the rule:

- **anion sites** are excluded (defined by `[scan].anion_species`)
- **all remaining sites** form the enumeration sublattice

This design makes the method general across crystals where doping occurs on a cation (or non-anion) sublattice.

The dopant counts are inferred directly from the input POSCAR by counting species on the enumeration sublattice. The host species is provided by `[doping].host_species`.

3.4.7 Combinatorics and Safety Limits

Raw configuration count

Given a sublattice with N sites and fixed dopant counts, the number of raw configurations (before symmetry reduction) grows combinatorially.

To avoid runaway enumeration, the workflow:

- estimates the raw count and enforces `[scan].max_enum`
- enforces a hard limit on symmetry-unique configurations via `[scan].max_unique`

If `mode = "auto"` and the estimated configuration count exceeds `[scan].max_enum`, the scan automatically switches to sampling mode.

If `mode = "exact"`, exceeding these limits stops the scan to prevent infeasible enumeration.

This protects against infeasible compositions and/or large supercells.

Maximum number of dopant species per scan

The current symmetry-unique enumerator is implemented explicitly for up to **three distinct dopant species** on the enumerated sublattice.

If the inferred dopant set contains more than three species, the scan raises an error.

This limit applies to the scanning enumeration procedure (not necessarily to other stages).

3.4.8 Symmetry Reduction

To avoid evaluating symmetry-equivalent configurations, the scan:

1. Constructs a **parent** structure where all sites of the enumeration sublattice are assigned to the host species.
2. Uses a space-group analysis (with tolerance `[scan].symprec`) to obtain symmetry operations.
3. Converts symmetry operations into **permutations of sublattice indices**.
4. For each dopant labeling, computes a canonical representation under these permutations.
5. Keeps only labelings with unique canonical keys.

This yields the set of symmetry-unique dopant configurations.

3.4.9 Sampling Mode

When `mode = "sample"` (or when `mode = "auto"` switches to sampling), the workflow generates configurations by random sampling instead of full combinatorial enumeration.

The algorithm:

1. randomly assigns dopants to sublattice sites
2. converts the labeling to a canonical symmetry representation
3. discards duplicates already encountered
4. accumulates unique configurations until a batch is formed
5. evaluates the batch using the selected machine-learning backend

Sampling is controlled by the parameters:

- `sample_budget` — maximum number of sampling attempts
- `sample_batch_size` — number of unique structures evaluated per batch
- `sample_patience` — early stopping when no better structures are found
- `sample_seed` — random seed for reproducibility
- `sample_max_saved` — maximum stored canonical keys to avoid duplicates

Sampling allows the scan to explore large configuration spaces that would otherwise be infeasible to enumerate.

3.4.10 Energy Evaluation and Parallel Execution

Each symmetry-unique configuration is evaluated using a **single-point** energy prediction with the selected ML backend.

Key points:

- structures are not relaxed at this stage
- only energies are predicted (fast screening)
- predictions are backend-dependent

Backend-specific behavior:

- `m3gnet`: - TensorFlow-based - GPU mode uses a single worker for stability
- `uma`: - PyTorch-based - supports CPU and GPU execution
- `mace`: - PyTorch-based foundation model - typically executed efficiently in single-process mode - model is loaded once and reused across all configurations
- `grace`: - PyTorch-based (if available) - performance depends on model implementation

Parallel execution:

- CPU mode uses multiprocessing (`n_workers`)
- GPU mode may restrict parallelism depending on backend
- worker processes are created using the `spawn` method for robustness

3.4.11 Selection of Top-k Candidates

The workflow keeps the `[scan].topk` configurations with the **lowest predicted** single-point energies and writes each to:

```
<structure_dir>/candidate_###/01_scan/POSCAR
<structure_dir>/candidate_###/01_scan/meta.json
```

A CSV ranking file is written to the structure directory:

```
<structure_dir>/ranking_scan.csv
```

The ranking CSV contains:

- candidate name
- rank (single-point)
- predicted single-point energy
- a short dopant-position signature

3.4.12 Reproducibility

For a fixed input structure and scan settings:

- enumeration mode produces deterministic results
- sampling mode is stochastic but reproducible when `sample_seed` is fixed
- inferred dopant counts
- symmetry analysis (controlled by `symprec`)
- enumeration ordering

- the ML model used for prediction

Parallel evaluation does not affect the final ranking, since candidates are selected based on energy values.

3.4.13 Outputs

For each processed structure folder:

- `candidate_*/01_scan/POSCAR`: POSCAR files of selected low-energy candidates
- `candidate_*/01_scan/meta.json`: metadata describing scan settings and counts
- `ranking_scan.csv`: ranked list of candidates with predicted energies
- `scan_summary.txt`: human-readable summary of the scan

3.4.14 Notes and Limitations

- This stage performs **single-point ML energy screening only**; it does not relax structures.
- Symmetry reduction depends on the tolerance `symprec`; too large values may merge distinct configurations, too small values may reduce symmetry detection.
- Enumeration can become infeasible for large sublattices and/or high dopant counts; limits `max_enum` and `max_unique` are enforced to prevent runaway runtime/memory.
- The current label enumerator supports up to **three distinct dopant species**.
- Predicted energies are surrogate ML values and should be interpreted as a ranking heuristic rather than absolute thermodynamic energies.
- Different backends provide different accuracy/speed trade-offs:
  - `m3gnet`: robust general-purpose model
  - `uma`: high-quality FAIR-Chem models (requires access permissions)
  - `mace`: fast foundation models suitable for large-scale screening
  - `grace`: experimental GNN-based models
- Sampling mode does not guarantee discovery of the global minimum but efficiently identifies low-energy candidates for subsequent relaxation.

3.5 3. Candidate Relaxation

3.5.1 Implementation

This stage is implemented in:

```
src/dopingflow/relax.py
```

The public entry point is:

```
run_relax(...)
```

3.5.2 Purpose

This stage performs **structural relaxation** of the low-energy candidates selected during the scanning stage.

For each candidate structure:

- atomic positions are relaxed
- lattice vectors are optimized together with atomic positions
- the final total energy is extracted
- relaxed structures are written to disk
- a relaxation ranking is generated

This stage refines the single-point screening results by allowing full geometry optimization with the selected machine-learning interatomic potential backend.

3.5.3 Inputs

This stage uses settings from the following sections of `input.toml`:

- `[structure]`: provides the output directory containing structure folders
- `[generate]`: provides the species ordering used when writing POSCAR files
- `[relax]`: controls backend, model, optimizer, convergence, execution mode, and skipping logic

The relaxation stage processes:

```
<outdir>/<structure_folder>/candidate_*/01_scan/POSCAR
```

3.5.4 Method Summary

For each structure folder inside `[structure].outdir`:

1. Detect all `candidate_*` subfolders.
2. For each candidate: a. Read `01_scan/POSCAR`. b. Initialize the selected ML backend. c. Relax the structure with the selected ASE optimizer. d. Write the relaxed structure to `02_relax/POSCAR`. e. Write relaxation metadata to `02_relax/meta.json`.
3. Rank candidates by relaxed total energy.
4. Write `ranking_relax.csv` per structure folder.

3.5.5 Relaxation Model

Structural relaxation is performed using a selectable machine-learning backend.

Supported backends:

- **M3GNet**
- **UMA**
- **MACE**
- **GRACE**

The backend is used as an ASE calculator, and relaxation is carried out using an ASE optimizer.

Supported optimizers:

- **BFGS**
- **LBFGS**
- **FIRE**
- **MDMin**

- **QuasiNewton**

The stopping criteria are:

- maximum atomic force below `[relax].fmax`
- or optimizer step count reaching `[relax].max_steps`

The relaxed degrees of freedom are controlled by `[relax].relax_mode`. By default, only atomic positions are relaxed. Cell relaxation can be enabled explicitly using modes such as `full`, `isotropic`, `volume`, `shape`, `xy`, or `cell_only`.

are allowed to relax.

3.5.6 Backend-Specific Notes

M3GNet

- Uses the pretrained M3GNet model through an ASE calculator
- Supports CPU and CUDA execution
- On GPU, the workflow uses a single effective worker for safe TensorFlow usage

UMA

- Uses FAIR-Chem pretrained models
- Requires both a selected `model` and a `task`
- Supports CPU and GPU execution depending on the environment

MACE

- Discovers the `mace_mp` foundation-model aliases supported by the installed MACE package, including `mh-0` and `mh-1` with MACE 0.3.14 or newer
- Accepts a local MACE checkpoint path through `model`
- Uses the existing `task` parameter as an optional multi-head model head
- Integrated through the ASE calculator interface
- Supports CPU and GPU execution

GRACE

- Uses GRACE foundation models through the tensorpotential interface
- Integrated through the ASE calculator interface
- Availability depends on the installed GRACE/tensorpotential environment

3.5.7 Parallel Execution

Relaxation is parallelized **over candidates** using multiprocessing.

Each worker process:

- initializes its own backend model or calculator
- applies thread limits where relevant
- relaxes one candidate at a time

The number of parallel workers is controlled by:

```
[relax]
n_workers = ...
```

Additional thread control parameters:

```
tf_threads
omp_threads
```

These settings are mainly relevant in CPU mode. When `device = "cuda"`, the workflow uses a single effective worker.

3.5.8 Ranking Logic

After all candidates are processed within a structure folder:

- Only successfully relaxed candidates (status `ok`) are ranked.
- Ranking is based on ascending relaxed total energy.

The resulting CSV file:

```
ranking_relax.csv
```

contains:

- candidate name
- relaxed rank
- relaxed energy
- original single-point rank
- original single-point energy
- signature
- status
- walltime
- convergence status
- final maximum force
- optimizer step count
- error message (if any)

3.5.9 Reproducibility and Skipping

Two levels of skipping are supported:

Folder-level skipping

If:

```
[relax].skip_if_done = true
```

and `ranking_relax.csv` already exists in a structure folder, the entire folder is skipped.

Candidate-level skipping

If:

```
[relax].skip_candidate_if_done = true
```

and:

```
candidate_*/02_relax/meta.json
candidate_*/02_relax/POSCAR
```

already exist, that individual candidate is skipped.

This enables safe restart of interrupted runs.

3.5.10 Outputs

For each candidate:

```
candidate_###/
    01_scan/
    02_relax/
        POSCAR
        meta.json
```

The relaxation metadata includes:

- backend
- model
- task
- optimizer
- relaxed total energy
- walltime
- species counts
- convergence target (fmax)
- maximum allowed optimizer steps
- actual optimizer step count
- final maximum force
- convergence status
- link to original scan metadata
- relax_mode
- cell_filter

For each structure folder:

```
ranking_relax.csv
```

which ranks relaxed candidates by energy.

3.5.11 Error Handling

If relaxation fails:

- an error is written to `02_relax/meta.json`
- the candidate is marked with status `fail`
- ranking proceeds with remaining successful candidates

Failures do not abort the entire structure folder.

3.5.12 Notes and Limitations

- Relaxation uses pretrained ML interatomic potentials rather than DFT.
- No charge states or external electric fields are included.
- No temperature or vibrational effects are considered.
- Energies are total energies predicted by the selected ML backend.
- Convergence depends on both the chosen `fmax` threshold and `max_steps`.
- Parallel performance depends on backend, hardware, and thread configuration.

3.6 4. Relaxed-Candidate Filtering

3.6.1 Implementation

This stage is implemented in:

```
src/dopingflow/filtering.py
```

The public entry point is:

```
run_filtering(...)
```

3.6.2 Purpose

This stage selects a **reduced set of relaxed candidates** for downstream property calculations by filtering the results of the relaxation stage.

It operates on the per-folder relaxation ranking produced in Step 03 and writes:

- a filtered ranking table
- a plain text list of selected candidate names

This stage is a lightweight post-processing step; it does not run any atomistic calculations.

3.6.3 Inputs

This stage uses settings from the following sections of `input.toml`:

- `[structure]`: provides the output directory containing structure folders.
- `[filter]`: defines the filtering strategy and thresholds.

It expects that Step 03 has already produced, in each structure folder:

```
ranking_relax.csv
```

3.6.4 Method Summary

For each structure folder inside `[structure].outdir`:

1. Read `ranking_relax.csv`.
2. Keep only candidates with `status == "ok"`.
3. Determine the minimum relaxed energy:
   - $E_{\min} = \min(E_{\text{relaxed}})$
4. Apply one of two filtering modes:
   - *window mode*: keep candidates within an energy window above $E_{\min}$
   - *top-n mode*: keep the lowest-energy `N` candidates
5. Write filtered outputs:
   - `ranking_relax_filtered.csv`
   - `selected_candidates.txt`

3.6.5 Filtering Modes

Window mode

If the filter mode is set to `window`, candidates are kept if:

$$E_{\text{relaxed}} - E_{\min} \leq \Delta E$$

where:

- $\Delta E = \text{window\_meV}/1000$ (converted from meV to eV)

This selects all structures that lie within a user-defined energy window above the best relaxed candidate.

Top-n mode

If the filter mode is set to `topn`, candidates are kept by selecting the first `max_candidates` entries after sorting by relaxed energy.

This guarantees a fixed number of candidates per structure folder (unless fewer successful relaxations exist).

3.6.6 Selection Basis

Filtering is based exclusively on:

- `energy_relaxed_eV` from `ranking_relax.csv`

The filter ignores candidates that:

- are missing required fields
- have non-numeric energies
- have `status` values other than `ok`

3.6.7 Outputs

For each structure folder, this stage writes:

Filtered ranking table

```
ranking_relax_filtered.csv
```

with columns including:

- `rank_filtered`: rank within the filtered set (starting at 1)
- `candidate`: candidate folder name
- `energy_relaxed_eV`: relaxed energy (eV)
- `delta_e_eV`: energy relative to the folder minimum
- provenance columns copied from the relaxation stage (e.g. scan rank/signature)
- `filter_mode`: a string describing the applied filter rule

Selected candidate list

```
selected_candidates.txt
```

This is a newline-separated list of candidate folder names, in the same order as the filtered ranking table.

3.6.8 Command-line Overrides and Forcing

The implementation supports runtime overrides that can force the filter behavior independently of the default TOML settings:

- overriding `window_meV` forces window mode
- overriding `topn` forces top-n mode

A force flag can be used to regenerate outputs even if they already exist.

3.6.9 Reproducibility and Skipping

If:

```
[filter].skip_if_done = true
```

and both output files already exist in a folder, that folder is skipped unless a force option is used.

Since this stage is pure file processing, its results are deterministic given the input `ranking_relax.csv` and filter parameters.

3.6.10 Notes and Limitations

- This stage performs **no new calculations**; it filters results from Step 03.
- Filtering is done independently per structure folder; it does not compare energies across different compositions/folders.
- Energy windows are applied relative to the minimum energy within each folder, not relative to a global minimum.

3.7 5. Bandgap Prediction

3.7.1 Implementation

This stage is implemented in:

```
src/dopingflow/bandgap.py
```

The public entry point is:

```
run_bandgap(...)
```

3.7.2 Purpose

This stage predicts the **electronic band gap** of relaxed candidate structures using a locally available **ALIGNN** model.

Band gaps are evaluated for candidates produced by the previous steps (typically after relaxation, and optionally after filtering) and written to:

- a per-folder summary CSV
- a per-candidate metadata record

This is a machine-learning inference step; no electronic-structure calculation is performed here.

3.7.3 Inputs

This stage uses settings from the following sections of `input.toml`:

- `[structure]`: provides the output directory containing structure folders.
- `[bandgap]`: defines graph-construction parameters and skipping behavior.

In addition, this stage requires an environment variable pointing to a local ALIGNN model directory:

```
ALIGNN_MODEL_DIR=/path/to/your/alignn_model_folder
```

3.7.4 Method Summary

For each structure folder inside `[structure].outdir`:

1. Determine the list of candidate structures to evaluate:
   a. If `selected_candidates.txt` exists, only those candidates are used.
   b. Otherwise, all `candidate_*/02_relax/POSCAR` files are used.
2. For each selected candidate:
   a. Read the relaxed `POSCAR` (from Step 03).
   b. Convert the structure to a graph representation (DGL graph) using a neighbor cutoff and maximum neighbor count.
   c. Run a forward pass of the ALIGNN model to obtain the predicted band gap.
   d. Write a per-candidate metadata file.
3. Write a per-folder summary CSV listing band gaps for all evaluated candidates.

3.7.5 Selection of Candidates

This stage supports two selection modes:

- **Filtered selection (recommended)**: If `selected_candidates.txt` exists in the folder, the band gap is computed only for those candidates. This integrates naturally with Step 04 filtering.
- **Fallback selection**: If no selection list is present, all relaxed candidates are used.

The relaxed structures are always taken from:

```
candidate_*/02_relax/POSCAR
```

3.7.6 Model and Graph Construction

ALIGNN model loading

The model is loaded from a local directory under `ALIGNN_MODEL_DIR`. The code searches for a folder containing a `config.json` and a checkpoint file (e.g. `checkpoint_*.pt` or `*.pt`), then loads:

- the model configuration (from `config.json`)
- the checkpoint weights

Graph parameters

For each structure, a graph is constructed using:

- `[bandgap].cutoff`: neighbor cutoff distance
- `[bandgap].max_neighbors`: maximum neighbors per atom

These parameters control how local environments are encoded for inference.

The DGL backend is set to PyTorch in this stage (via `DGLBACKEND=pytorch`), which must be configured before importing DGL/ALIGNN.

3.7.7 Outputs

Per-folder summary

For each structure folder, this stage writes:

```
bandgap_alignn_summary.csv
```

This file contains:

- `candidate`: candidate directory name
- `bandgap_eV_ALIGNN_MBJ`: predicted band gap (eV)

The CSV is sorted by band gap (ascending) for convenience.

Per-candidate metadata

For each evaluated candidate, this stage writes:

```
candidate_XXX/03_band/meta.json
```

This metadata includes:

- predicted band gap value

- model provenance (model directory, config path, checkpoint path)
- input structure path
- structure size and composition
- graph parameters used for inference
- walltime for prediction

3.7.8 Reproducibility and Skipping

If:

```
[bandgap].skip_if_done = true
```

and `bandgap_alignn_summary.csv` already exists for a folder, that folder is skipped.

Given the same relaxed POSCARs, model checkpoint, and graph parameters, this stage is deterministic.

3.7.9 Notes and Limitations

- Predicted band gaps depend on the chosen ALIGNN model and its training domain.
- Band gaps are ML predictions and should not be interpreted as DFT-quality values unless the chosen model has been validated for the target material class.
- This stage assumes candidate structures are already relaxed; it does not perform geometry optimization.

3.8 6. Formation Energy Evaluation

3.8.1 Implementation

This stage is implemented in:

```
src/dopingflow/formation.py
```

The public entry point is:

```
run_formation(...)
```

3.8.2 Purpose

This stage computes the **formation energy** of relaxed doped structures using reference energies constructed in Step 00.

It combines:

- the relaxed total energy of each doped candidate structure (E_{doped})
- the relaxed total energy of the pristine supercell (E_{pristine})
- elemental chemical potentials (μ_i) for host and dopant species

Formation energies are written per composition folder to:

- `formation_energies.csv` (summary table)
- `candidate_*/04_formation/meta.json` (per-candidate provenance)

3.8.3 Inputs

This stage uses settings from the following sections of `input.toml`:

- `[structure]`: provides the output directory containing structure folders.
- `[doping]`: defines the substitution host species.
- `[scan]`: provides the anion species list used to identify dopants.
- `[formation]`: controls skipping, normalization, and optional relative columns.

It also requires the reference-energy JSON from Step 00:

```
reference_structures/reference_energies.json
```

3.8.4 Formation Energy Framework

Substitutional doping model

The workflow assumes substitutional doping on a host sublattice. Dopants are identified as all species that are:

- not equal to the host species (`[doping].host_species`), and
- not in the anion list (`[scan].anion_species`)

The set of dopant counts n_i is extracted from each candidate POSCAR.

Formation energy definition

The formation energy is defined as:

$$E_{\text{form}} = E_{\text{doped}} - E_{\text{pristine}} + \sum_i n_i \left(\mu_{\text{host}} - \mu_i\right)$$

where:

- E_{doped} is the relaxed total energy of the doped supercell
- E_{pristine} is the relaxed total energy of the pristine supercell
- μ_{host} is the host chemical potential (per atom)
- μ_i is the dopant chemical potential (per atom)
- n_i is the number of dopant atoms of species i in the supercell

This corresponds to replacing n_i host atoms by n_i dopant atoms for each dopant species i, while keeping the same supercell size.

Reference energies are taken from Step 00 and must be consistent with the supercell size and host species used here.

3.8.5 Method Summary

For each structure folder inside `[structure].outdir`:

1. Load the reference data from:

   ```
   reference_structures/reference_energies.json
   ```

   extracting E_{pristine} and chemical potentials μ_i.

2. Determine which candidates to evaluate:

   a. If `selected_candidates.txt` exists, only those candidates are used.

   b. Otherwise, all `candidate_*/02_relax/POSCAR` files are used.

3. For each selected candidate:
   a. Read the relaxed energy E_{doped} from `candidate_*/02_relax/meta.json`.
   b. Read species counts from `candidate_*/02_relax/POSCAR` and infer dopant counts under the substitutional model.
   c. Evaluate E_{form} using the equation above.
   d. Apply the requested normalization (see below).
   e. Write `candidate_*/04_formation/meta.json`.
4. Write `formation_energies.csv` in the folder, sorted by total formation energy.

3.8.6 Normalization Options

This stage supports three reporting modes controlled by:

```
[formation]
normalize = "total" | "per_dopant" | "per_host"
```

The internal formation energy is always computed as a **total supercell energy** (E_{form} in eV). The reported value can be:

- `total`: report E_{form} in eV (no normalization)
- `per_dopant` (default): report $E_{\mathrm{form}}/N_{\mathrm{dop}}$, where $N_{\mathrm{dop}} = \sum_i n_i$ is the total number of dopant atoms
- `per_host`: report $E_{\mathrm{form}}/N_{\mathrm{atoms}}$, where N_{atoms} is the total number of atoms in the pristine supercell (as stored in the reference JSON)

Note: `per_host` currently uses the total number of atoms in the pristine supercell. If you later want normalization per *host-sublattice* site, that quantity can be stored explicitly in the reference JSON and used here.

3.8.7 Multiple Oxide References and Relative Values

In oxide mode, every binary oxide listed for a dopant is evaluated. Co-doped candidates use the Cartesian product of the available oxide choices, producing deterministic scenario labels such as `Sb2O3__TiO2`.

Relative output is configured without adding another TOML section:

```
[formation]
relative_enabled = true
endpoint_x = "auto"
```

The oxide chemical potentials and the atom-balanced mixing reaction already reference the candidate to its host-oxide/dopant-oxide tie-line. Consequently, the relative per-cation values equal their corresponding oxide-referenced formation and mixing values; collection does not apply a second endpoint subtraction. `endpoint_x = "auto"` records the pure oxide endmember ($x = 1$).

For co-doping, endpoint provenance includes both the endpoint energy for each dopant and the composition-weighted correction $\sum_i x_i E_{\mathrm{endpoint},i}$.

3.8.8 Outputs

Per-folder summary

For each structure folder, this stage writes:

```
formation_energies.csv
```

Base columns:

- `candidate`: candidate directory name
- `E_doped_eV`: relaxed total energy of the doped candidate
- `n_dopant_atoms`: total dopant atoms N_{dop}
- `dopant_counts`: compact dopant count string (e.g. `Sb:2;Zr:1`)
- `x_dopant`: actual dopant fraction on the cation sublattice
- `reference_mode`: `metal` or `oxide`

Each reference scenario then contributes wide columns with a `__<scenario>` suffix, including formation energies, mixing energies, relative values, oxide endpoint energies, reaction text, and JSON endpoint provenance. Rows remain one candidate per row and are sorted by relaxed doped energy.

Per-candidate metadata

For each evaluated candidate, this stage writes:

```
candidate_XXX/04_formation/meta.json
```

This file includes:

- full formation-energy definition string from the reference JSON
- E_{doped}, E_{pristine}
- chemical potentials used for the involved species
- inferred dopant counts
- total formation energy and the reported normalized value
- the full `reference_results` mapping for all oxide scenarios
- the primary reference label used for backward-compatible top-level fields
- relative-energy and oxide-endpoint provenance

3.8.9 Reproducibility and Skipping

If:

```
[formation].skip_if_done = true
```

and `formation_energies.csv` already exists for a folder, that folder is skipped.

Given unchanged relaxed energies, POSCARs, reference JSON, and configuration, this stage is deterministic.

3.8.10 Notes and Limitations

- This stage assumes substitutional doping and uses a simple species-based dopant identification rule (host vs anions vs dopants).
- No charged-defect corrections, finite-size corrections, entropy terms, or competing-phase chemical potential bounds are included.
- The absolute values depend on the reference energies and the chosen bulk phases used to define μ_i.

3.9 7. Database Collection

3.9.1 Implementation

This stage is implemented in:

```
src/dopingflow/collect.py
```

The public entry point is:

```
run_collect(...)
```

3.9.2 Purpose

This stage consolidates results from previous workflow stages into a **single flat CSV** that can be used as a lightweight database for downstream analysis, plotting, and reporting.

The database is written to the workflow root as:

```
results_database.csv
```

Only **filtered / selected candidates** are included (strict policy), so the database represents the subset of structures that passed your selection criteria.

3.9.3 Inputs

This stage uses settings from two sections of `input.toml`:

- `[structure]`: provides the workflow output directory containing composition folders.
- `[database]`: controls skipping/overwriting of the database CSV.

It expects the standard workflow outputs inside each composition folder (if present):

Composition-level (folder-level)

- `metadata.json` (from Step 01 generation)
- `selected_candidates.txt` (from Step 04 filtering) **preferred**
- `ranking_relax_filtered.csv` (from Step 04 filtering) **fallback**
- `ranking_scan.csv` (from Step 02 scanning)
- `bandgap_alignn_summary.csv` (from Step 05 bandgap prediction)
- `formation_energies.csv` (from Step 06 formation energies)

Candidate-level

- `candidate_*/02_relax/meta.json` (from Step 03 relaxation)
- `candidate_*/04_formation/meta.json` (preferred formation source)

The per-candidate formation metadata is preferred because it retains the full multi-reference result mapping. `formation_energies.csv` is used as a backward-compatible fallback.

3.9.4 Selection Policy

This stage follows a **strict selection rule**: it will never include unfiltered candidates.

Selection priority within each composition folder:

1. If `selected_candidates.txt` exists: use exactly those candidate names (one per line).
2. Else, if `ranking_relax_filtered.csv` exists: include the candidates listed there.
3. Else: the folder is skipped (no candidates are collected).

This ensures that the database represents only the candidates you explicitly kept after filtering.

3.9.5 Method Summary

For each composition folder inside `[structure].outdir`:

1. Determine the candidate list using the selection policy above.
2. Load composition metadata from `metadata.json` (if available).
3. Load per-stage summary tables (if available):
   - scan ranking table
   - filtered ranking table
   - bandgap summary table
   - formation energy table
4. For each selected candidate:
   a. Read relaxed-energy metadata from `candidate_*/02_relax/meta.json` (if available).
   b. Read formation metadata from `candidate_*/04_formation/meta.json`.
   c. Flatten every reference scenario into wide `__<scenario>` columns.
   d. Combine composition-level and candidate-level information into one row.
5. Write all rows to:

```
results_database.csv
```

3.9.6 Database Schema

The output database contains the following columns:

Composition-level fields

- `composition_tag`: Folder name of the composition (effective tag used by the generator).
- `requested_index`: Index of the composition in the generator loop (if available).
- `requested_pct_json`: JSON string of requested dopant percentages (if available).

- `effective_pct_json`: JSON string of effective dopant percentages after rounding (if available).
- `rounded_counts_json`: JSON string of rounded integer substitution counts (if available).
- `host_species`: Host species used for substitution (from generation metadata).
- `n_host`: Number of host sites in the supercell (from generation metadata).
- `supercell_json`: JSON string of the supercell dimensions (if available).

Candidate-level identity

- `candidate`: Candidate folder name (e.g., `candidate_001`).
- `candidate_path`: Absolute path to the candidate folder (useful for linking back to files).

Relax (filtered) fields

- `rank_relax_filtered`: Rank within the filtered set (if available).
- `E_relaxed_eV_filtered`: Relaxed energy read from the filtered ranking table (if available).

Scan fields

- `rank_scan`: Single-point rank from scanning stage (if available).
- `E_scan_eV`: Single-point energy from scanning stage (if available).

Relax (meta) fields

- `E_relaxed_eV`: Relaxed energy extracted from `candidate_*/02_relax/meta.json` (if available).

Bandgap fields

- `bandgap_eV`: Predicted bandgap value from the bandgap summary (if available).

Formation-energy fields

- `E_form_eV_total`: Total formation energy in eV (if available).
- `E_form_norm`: Normalized formation energy (as written by Step 06, depends on your normalization mode).
- `n_dopant_atoms`: Total dopant atoms used in the candidate (if available).
- `dopant_counts`: Compact dopant-count string from Step 06 (if available).
- `primary_reference_label`: Reference scenario used for the backward-compatible unsuffixed fields.

Multi-reference fields

Every reference scenario is encoded as a column suffix. Examples include:

- `E_form_eV_per_cation__Sb2O3`
- `E_mix_eV_per_cation__Sb2O3`
- `E_form_rel_eV_per_cation__Sb2O3`
- `oxide_endpoint_eV_per_cation__Sb2O3`
- `oxide_endpoint_by_dopant_json__Sb2O3`
- `oxide_references_json__Sb2O3`

Co-doped scenarios retain all oxide names in a deterministic suffix, for example `E_mix_eV_per_cation__Sb2O3__TiO2`. Formation-stage relative columns are authoritative and are preserved during collection.

3.9.7 Outputs

This stage writes one file in the workflow root:

```
results_database.csv
```

Each row corresponds to **one selected candidate** in **one composition folder**.

3.9.8 Reproducibility and Skipping

If:

```
[database].skip_if_done = true
```

and `results_database.csv` already exists, the stage is skipped.

Set `skip_if_done = false` to overwrite and regenerate the database.

3.9.9 Notes and Limitations

- The database is intentionally flat and file-based; it is designed for quick use in pandas, spreadsheets, or plotting scripts.
- Missing upstream files are handled gracefully: columns are left empty/`None` when a stage output is unavailable.
- Only the filtered/selected candidate subset is included by design. If you later want a "full database" including all candidates, the selection policy can be relaxed as an optional mode.

3.10 Restricted One-Dimensional Alloy Hull

The `alloy-hull` stage builds a **restricted one-dimensional convex hull** for a single substitutional alloy path. For example, for Sb substitution in SnO2 it uses the fixed-composition line:

$$\mathrm{Sn}_{1-x}\mathrm{Sb}_x\mathrm{O}_2.$$

This stage is different from `phase-diagram`:

- `alloy-hull` compares only SnO2-type host, the selected Sb substitutional candidates, and an optional stoichiometrically compatible endpoint such as SbO2.
- `phase-diagram` is the full multicomponent Sn-Sb-O stability calculation; it may decompose a candidate into phases outside the Sn_(1-x)Sb_xO2 line.

3.10.1 Energy used for the hull

The stage uses direct relaxed total energies, normalized per cation:

$$e_\sigma = \frac{E_\sigma}{N_{\mathrm{Sn}} + N_{\mathrm{Sb}}}.$$

At each composition, only the lowest-energy candidate is retained. The lower convex envelope is then constructed from these minima, the relaxed host at `x = 0`, and the endpoint reference at `x = 1` when one is available.

For a candidate at `x` lying between two neighbouring hull vertices `(x_1, e_1)` and `(x_2, e_2)`, the stage computes:

$$e_{\mathrm{hull}}(x) = e_1 + \frac{x - x_1}{x_2 - x_1}(e_2 - e_1),$$

and reports:

$$E_{\text{above 1D hull}}(x) = e_\sigma(x) - e_{\mathrm{hull}}(x).$$

This is reference-independent for a fixed alloy line: adding any term linear in `x` to every energy, such as the oxide-reference or oxygen-chemical- potential contribution, changes the energy zero and slopes but does not change the hull vertices or energy above the restricted 1D hull.

3.10.2 Configuration

The section is optional. The defaults are appropriate for a single-dopant workflow:

```
[alloy_hull]
dopant = "auto"
endpoint_reference = "auto"
```

`dopant = "auto"` detects the unique dopant in the result database. When multiple dopants are present, specify one explicitly, for example:

```
[alloy_hull]
dopant = "Sb"
endpoint_reference = "SbO2"
```

`endpoint_reference = "auto"` searches the cached oxide references for a binary dopant oxide with the same anion-to-cation ratio as the host line. For SnO2 doped with Sb, it selects SbO2. Set `endpoint_reference = "none"` to build a hull only over the sampled range; this is useful when no physical x = 1 endpoint has been calculated or cached.

3.10.3 Run

Run after collection:

```
dopingflow collect -c input.toml
dopingflow alloy-hull -c input.toml
```

The `run-all` command also includes this stage between `collect` and `phase-diagram`.

3.10.4 Outputs

The stage writes:

- `alloy_hull_results.csv`: one row per relaxed candidate, including `energy_per_cation_eV`, `hull_energy_1d_eV_per_cation`, `energy_above_1d_hull_eV_per_cation`, `on_1d_hull`, and the predicted 1D decomposition between neighbouring hull vertices.
- `alloy_hull_vertices.csv`: the actual lower-hull vertices, including host and endpoint reference points.
- `alloy_hull_summary.json`: stage metadata and selected endpoint reference.

The same 1D hull fields are also appended to `results_database.csv` so they can be used directly in Jupyter plotting scripts.

3.11 Phase Diagram and Energy Above Hull

The phase-diagram step evaluates relaxed candidates with pymatgen's `PhaseDiagram` class. It builds a separate diagram for every exact chemical system represented in `results_database.csv`. Examples include `O-Sb-Sn`, `Ce-O-Sb-Sn`, and `In-O-Sb-Sn`.

For a given system, the hull includes:

- reference phases whose elements are a subset of that system;
- candidates from the exact system;
- lower-dimensional candidates whose elements are a subset of that system.

Only candidates belonging to the exact system are reported in that system's CSV. This prevents unrelated dopants from being combined in one artificial high-dimensional hull.

3.11.1 Terminal References

A closed phase diagram requires an elemental terminal entry for every element in the system. `O2` supplies the oxygen terminal because its composition contains only oxygen. Every metal must be listed in `metal_ref` and have a corresponding POSCAR available when `refs-build` is run, including when `reference_mode = "oxide"`.

The step stops with a descriptive error if a terminal is missing.

3.11.2 Configuration

All settings remain in the existing `[phase_diagram]` section:

```
[phase_diagram]
skip_if_done = true
stable_threshold_eV_per_atom = 0.05
```

`skip_if_done` returns the existing combined output without rebuilding the diagrams. `stable_threshold_eV_per_atom` controls the boolean `stable` column and must be non-negative.

3.11.3 Energy Above Hull

For each candidate:

$$E_{\text{above hull}} = E_{\text{candidate}} - E_{\text{hull}}$$

where E_{hull} is the lowest-energy combination of the available entries with the same overall composition.

3.11.4 Outputs

The combined output is:

`phase_diagram_results.csv`

Individual systems are also written under `phase_diagrams/`:

```
phase_diagrams/phase_diagram_O-Sb-Sn.csv
phase_diagrams/phase_diagram_Ce-O-Sb-Sn.csv
```

Columns include:

- `chemical_system`
- `candidate` and `composition_tag`

- formula
- energy_total_eV and energy_per_atom_eV
- energy_above_hull_eV_per_atom
- stable
- decomposition

3.12 Oxygen-Vacancy Workflow

dopingflow vacancies -c input.toml is the only public vacancy command. It discovers filtered relaxed parents, determines charge-based vacancy counts, enumerates symmetry-distinct arrangements, evaluates single-point ML energies, selects the lowest-energy structures independently at each vacancy count, relaxes that top-k set, reranks it, and writes parent-level and global summaries.

3.12.1 Configuration

All settings live in one flat [vacancies] table. Screening and relaxation share one resolved backend, model, task and device combination. The backend may be M3GNet, UMA, MACE, or GRACE.

```
[vacancies]
enabled = true
parent_source = "selected_candidates"
include_parent_reference = true
skip_if_done = true
resume = true

count_mode = "all_reachable"
host_species = "Sn"
host_oxidation_state = 4
vacancy_species = "O"
vacancy_compensation_charge = 2
oxidation_state_elements = ["Sb", "Nb"]
oxidation_state_values = [[3, 5], [5]]

extra_vacancies = 0
max_vacancies_cap = 8
symprec = 1.0e-3
angle_tolerance = 5.0
mapping_tolerance = 1.0
enumeration_mode = "auto"
max_exact_raw_configs = 300000
max_exact_unique_configs = 100000
sample_budget = 20000
sample_batch_size = 256
sample_patience = 4000
sample_seed = 42
sample_max_saved = 50000
minimum_vacancy_distance = 0.0

backend = "mace"
model = "medium-mpa-0"
task = ""
```

(continues on next page)

(continued from previous page)

```
device = "cuda"
gpu_id = 0
n_workers = 1
tf_threads = 1
omp_threads = 1
chunksize = 25

topk_per_vacancy_count = 15
energy_normalization = "per_vacancy"
optimizer = "bfgs"
fmax = 0.05
max_steps = 300
relax_mode = "atoms"
cell_filter = "frechet"
```

3.12.2 Formal charge and vacancy range

Actual dopant counts are read from each selected relaxed parent, not inferred from requested percentages. A dopant in formal state z_d replacing a host in state z_h contributes $z_d - z_h$. Contributions from all co-dopants are combined before a vacancy count is calculated. Mixed states are represented as population counts, avoiding permutations over individual atoms.

For every reachable negative total charge ΔQ, the upper relevant count is `ceil(-delta_Q / vacancy_compensation_charge)`. The workflow explores the continuous range from one through the largest upper count, then applies `extra_vacancies`, `max_vacancies_cap`, and the number of available oxygen sites. Residual charge is retained for every scenario and count; exact compensation is marked only when it is zero.

Eight Sb3+ atoms on Sn4+ sites give `8 × (3 - 4) = -8` and therefore generate counts 1, 2, 3, and 4. Four Sb3+ plus four Nb5+ give `-4 + 4 = 0` and no defective count unless `extra_vacancies` is positive. For `delta_Q = -3`, one vacancy has residual charge -1 and two vacancies have residual charge +1; both occur because the generated range ends at `ceil(3/2) = 2`.

These are formal assumptions defining the search space. They do not establish actual oxidation states. Later electronic-structure work may require Bader charges, projected densities of states, magnetic moments, local coordination, and charge-density differences.

3.12.3 Parents, symmetry, and enumeration

The default source is each composition's `selected_candidates.txt`. The corresponding `01_scan/POSCAR` supplies symmetry and the selected `02_relax/POSCAR` supplies coordinates. Sites are mapped by species and nearest periodic position; unreliable mappings fail explicitly.

To process an existing collection containing many composition subdirectories, select a directory source. The directory is resolved relative to `input.toml` unless it is absolute. Every immediate composition subdirectory may contain its own `selected_candidates.txt`:

```
parent_source = "directory"
parent_directory = "Mn-Nb-Ta-Ce-Ru-In-Sn-Sb"
```

The global `vacancies_database.csv/json` files are then written into that parent directory. Every dopant found in any processed parent, including Ce in this example, must have an entry in the flat oxidation-state arrays.

Vacancy occupancy vectors are canonicalized under the parent's symmetry permutations. `exact` enumerates all combinations and records exact orbit degeneracy. `sample` uses `sample_seed`, deduplicates canonical keys, and sets degeneracy to null because it is not exact. `auto` selects exact mode below `max_exact_raw_configs` and restarts in

sampled mode if the exact unique limit is exceeded. `minimum_vacancy_distance` filters close periodic pairs.

3.12.4 Screening, relaxation, and interpretation

The parent and every defective structure use the same common calculator factory. Single-point energies and relaxed energies are ranked only among structures having the same parent and vacancy count. The parent does not consume a top-k slot. Its prior relaxed energy is reused only when metadata prove calculator and relaxation compatibility; otherwise a consistency relaxation is written below the vacancy output without modifying the original parent.

Raw totals across different vacancy counts are not defect formation energies, because the structures contain different numbers of oxygen atoms. Thermodynamic comparison requires an oxygen chemical potential, for example `E_defect - E_parent + n * mu_O`. This workflow does not calculate equilibrium oxygen deficiency or finite-temperature defect thermodynamics.

3.12.5 Outputs and resume

Each parent receives `05_vacancies/` containing `parent_reference/`, `vacancy_counts.csv/json`, `vacancy_results.csv/json`, and one `V_O_NN/` group per count. Each configuration retains `00_generate`, `01_scan`, and `02_relax` provenance. The structure output root also receives `vacancies_database.csv` and `vacancies_database.json`. These remain separate from normal formation and phase-diagram databases.

Configuration and source checksums form a stable fingerprint. Compatible complete parents are skipped, compatible internal scan/relax metadata are resumed, and incompatible results are recomputed rather than silently reused.

Run directly or append the one run-all step:

```
dopingflow vacancies -c input.toml
dopingflow run-all -c input.toml --until vacancies
dopingflow run-all -c input.toml --only vacancies
```

3.13 Surface Generation and Relaxation

3.13.1 Overview

The surface stage extends the bulk workflow by constructing slab models from selected relaxed bulk candidates and optionally relaxing those slabs.

This stage operates **after** bulk structure generation, filtering, bandgap prediction, and formation energy evaluation.

The workflow consists of:

1. Selecting candidate structures from a database
2. Generating slab geometries for chosen surface orientations
3. Enumerating surface terminations
4. Optionally fixing part of the slab
5. Optionally relaxing the slab using a machine-learning backend
6. Writing structures and metadata

3.13.2 Input Data

The surface stage reads candidate data from a CSV database, typically:

```
results_database.csv
```

Each row corresponds to a bulk candidate and contains:

- `composition_tag`
- `candidate` (e.g. `candidate_001`)
- `candidate_path`
- `E_form_norm`
- `bandgap_eV`

The corresponding relaxed bulk structure is loaded from:

```
candidate_path / 02_relax / POSCAR
```

3.13.3 Candidate Selection

Selection is performed in two steps:

1. **Composition filtering**

   The dataset is first restricted using:

   - `composition_tag`
   - or `composition_tags`

2. **Selection mode**

   Within the selected composition subset, candidates are filtered using:

   - ID selection (`candidate_id`)
   - multiple IDs (`candidate_ids`)
   - ranking-based selection
   - numeric filters (formation energy and bandgap)

This two-step strategy ensures that comparisons are always made within the same chemical composition.

3.13.4 Slab Generation

For each selected bulk structure, slabs are generated using `pymatgen.core.surface.SlabGenerator`.

The user can choose between two modes:

Explicit orientation

The Miller indices are directly provided:

```
miller_list = [[1, 0, 0], [1, 1, 0], [1, 1, 1]]
```

Automatic orientation

All Miller indices up to `max_miller` are generated, and the first `max_orientations` are kept.

Slab construction

The slab is built according to:

- minimum slab thickness
- minimum vacuum thickness
- optional lattice reduction
- optional primitive reduction
- optional lattice reorientation

Important:

If `in_unit_planes = false`, both slab and vacuum thickness are interpreted in Ångström.

Orthogonalization

If `orthogonal_c = true`, the slab is transformed so that:

- the `c` vector is perpendicular to the surface plane
- `a` and `b` span the surface

This is recommended for:

- clean POSCAR output
- stable relaxation behavior
- easier interpretation of slab thickness

3.13.5 Surface Terminations

Each orientation can produce multiple terminations.

The workflow uses:

```
slabs = SlabGenerator(...).get_slabs()
```

Each slab corresponds to a different termination of the same surface.

The user controls:

- whether to keep all terminations
- or only the first one
- the maximum number of terminations per orientation

No symmetry-based deduplication is applied by default, ensuring that all physically distinct terminations are preserved.

3.13.6 Atom Ordering

Before writing structures, atoms are reordered according to:

```
[generate].poscar_order
```

Within each species, atoms are sorted by:

1. fractional z
2. fractional y

3. fractional x

This ensures:

- clean and consistent POSCAR files
- compatibility with VASP and post-processing tools

3.13.7 Fixed Atoms (Selective Dynamics)

The surface stage supports fixing part of the slab.

Two strategies are available:

Layer-based fixing

Atoms are grouped into layers based on their Cartesian z-coordinate.

Grouping is controlled by a tolerance:

```
fix_layer_tolerance_A
```

The user selects:

- bottom layers
- or middle layers

Thickness-based fixing

Atoms are selected based on their z-position:

- bottom region: atoms within a thickness from the minimum z
- middle region: atoms around the slab center

Output behavior

Fixed atoms are written in the same `POSCAR` file using VASP selective dynamics:

- fixed atoms: `F F F`
- free atoms: `T T T`

Important:

These flags are not only written to file but are also enforced during relaxation using ASE constraints.

3.13.8 Surface Relaxation

If enabled, slabs are relaxed using the same abstraction layer as bulk relaxation.

Workflow

1. Convert the slab to ASE atoms
2. Attach the ML calculator
3. Apply `FixAtoms` constraint (if needed)
4. Run ASE optimizer
5. Convert back to pymatgen structure

Backends

The relaxation uses the same backend system as the bulk stage:

- M3GNet
- ALIGNN (if available)
- other supported ML potentials

Runtime configuration includes:

- device (CPU or CUDA)
- GPU ID
- threading settings

Optimizer

Supported optimizers:

- BFGS
- LBFGS
- FIRE
- MDMin
- QuasiNewton

The optimizer runs until:

- maximum force < `surface_fmax`
- or maximum steps reached

Constraints

Fixed atoms are enforced using:

```
ase.constraints.FixAtoms
```

This ensures:

- atoms marked as fixed do not move during relaxation
- constraints are physically enforced, not only written in POSCAR

3.13.9 Surface Energy

The surface stage can evaluate the surface energy of each slab when a slab energy is available (typically after surface relaxation).

Definition

The surface energy is computed using the symmetric slab expression:

```
:math:`\gamma = \frac{E_{\mathrm{slab}} - n E_{\mathrm{bulk}}}{2A}`
```

where:

- `E_slab` is the slab total energy

- `E_bulk` is the relaxed bulk energy of the parent candidate
- `n` is the number of bulk-equivalent units contained in the slab
- `A` is the surface area
- the factor of 2 accounts for the two exposed surfaces

Bulk reference

The bulk energy is taken from the relaxed candidate structure selected from the workflow database.

This ensures consistency between bulk and surface calculations for the same doped composition.

Slab energy

Surface energy is computed only when a slab energy is available.

In the current implementation, this corresponds to:

- relaxed slab energy when `relax_surface = true`

If no slab energy is available, surface energy is not computed.

Bulk equivalence factor

The factor `n` is determined automatically from the composition of the slab and the parent bulk.

For each species, the ratio:

```
N_slab / N_bulk
```

is computed.

If all species share the same ratio within numerical tolerance, the slab is considered proportional to the bulk composition and this value defines `n`.

If the ratios differ between species, the slab is not considered proportional and surface energy is not assigned.

Stoichiometry requirement

Surface energy is only computed for slabs whose composition is proportional to the bulk.

This includes:

- stoichiometric slabs
- symmetric cleavage surfaces

Surface energy is not computed for:

- non-stoichiometric terminations
- slabs missing bulk species
- slabs containing additional species

Status reporting

Each slab is assigned a status flag:

- `ok` — surface energy successfully computed
- `missing_slab_energy` — slab energy not available
- `missing_bulk_energy` — bulk energy not available

- `not_computable_not_proportional` — composition mismatch
- `not_computable_missing_species`
- `not_computable_extra_species`
- `failed` — numerical or runtime failure

Units

Surface energy is reported in:

- eV/Å^2
- J/m^2

The conversion used is:

```
1 eV/Å^2 = 16.02176634 J/m^2
```

3.13.10 Outputs

Each slab produces a directory:

```
composition_tag / candidate / hkl / termination
```

Example:

```
Sb50/candidate_001/hkl_1_0_0/term_001/
```

Files

POSCAR
: Generated slab (with optional selective dynamics)

CONTCAR
: Relaxed slab structure (if relaxation is enabled)

surface_relax.log
: Optimizer log

surface_relax.traj
: ASE trajectory

surface_relax.json
: Relaxation metadata

meta.json
: General slab metadata

3.13.11 Metadata

Each slab has a `meta.json` file containing:

- composition and candidate information
- Miller index and termination ID
- number of atoms
- surface area

- slab and vacuum thickness estimates
- fixing parameters
- relaxation settings and results
- surface energy (when available)
- surface energy status flag
- bulk-equivalent scaling factor

This metadata is also aggregated into:

```
surface_summary.csv
```

3.13.12 Design Considerations

The surface stage is designed to:

- remain fully decoupled from bulk workflow stages
- allow flexible selection of candidates
- preserve all physically meaningful terminations
- support reproducible slab generation
- reuse the same backend abstraction for relaxation

This ensures consistency between:

- bulk relaxation
- surface relaxation
- downstream surface simulations

CHAPTER

FOUR

EXAMPLES

4.1 Explicit Example — Single Target Composition

4.1.1 Purpose

This example generates and evaluates **one explicit co-doped composition**.

Use this mode when: - You already know the exact dopant percentages. - You want exactly one generated structure per composition. - You want the full workflow executed (00–07).

4.1.2 Workflow

Run the complete pipeline:

```
dopingflow run-all -c input.toml
```

4.1.3 Required Files

The working directory must contain:

```
input.toml
reference_structures/
    base.POSCAR
    Sn.POSCAR
    Sb.POSCAR
    Zr.POSCAR
```

- `base.POSCAR`: pristine unit cell.
- Elemental `*.POSCAR`: bulk reference structures for chemical potentials.

4.1.4 Example input.toml

```
[structure]
outdir = "random_structures"

[references]
reference_mode = "metal"
host = "SnO2"
host_dir = "reference_structures/"
supercell = [ 5, 2, 1]
metals_ref = [ "Zr","Sb","Sn"]
```

(continues on next page)

(continued from previous page)

```
oxides_dir = "reference_structures/"
fmax = 0.02
max_steps = 300
tf_threads = 1
omp_threads = 1
device = "cpu"
gpu_id = 0
backend = "m3gnet"
model = "default"
task = ""
optimizer = "bfgs"
skip_if_done = false

[doping]
mode = "explicit"
host_species = "Sn"
compositions = [
{ Sb = 5.0, Zr = 5.0 }
]

[generate]
poscar_order = ["Zr","Sb","Sn","O"]
seed_base = 12345
clean_outdir = true

[scan]
backend = "m3gnet"
model = "default"
task = ""
poscar_in = "POSCAR"
topk = 10
symprec = 1e-3
max_enum = 10
n_workers = 4
chunksize = 10
anion_species = ["O"]
max_unique = 50000
skip_if_done = false
mode = "auto"
sample_budget = 5000
sample_batch_size = 64
sample_patience = 1000
sample_seed = 42
sample_max_saved = 10000
device = "cpu"

[relax]
backend = "m3gnet"
model = "default"
task = ""
optimizer = "bfgs"
fmax = 0.05
```

(continues on next page)

(continued from previous page)

```
max_steps = 300
relax_mode = "atoms"
cell_filter = "frechet"
n_workers = 6
tf_threads = 1
omp_threads = 1
skip_if_done = false
skip_candidate_if_done = false
device = "cpu"
gpu_id = 0

[filter]
mode = "window"
window_meV = 50.0
max_candidates = 12
skip_if_done = false

[bandgap]
skip_if_done = false
cutoff = 8.0
max_neighbors = 12
n_workers = 4
device = "cpu"

[formation]
skip_if_done = true
normalize = "per_dopant"

[database]
```

4.1.5 Outputs

- `reference_structures/reference_energies.json`
- `random_structures/<composition>/` - `ranking_scan.csv` - `ranking_relax.csv` - `ranking_relax_filtered.csv` - `bandgap_alignn_summary.csv` - `formation_energies.csv`
- `results_database.csv`

4.2 Explicit Example — Single Target Composition using Oxide References

4.2.1 Purpose

This example generates and evaluates **one explicit co-doped composition using oxides references**.

Use this mode when: - You already know the exact dopant percentages. - You want exactly one generated structure per composition. - You want the full workflow executed (00–07).

4.2.2 Workflow

Run the complete pipeline:

```
dopingflow run-all -c input.toml
```

4.2.3 Required Files

The working directory must contain:

```
input.toml
reference_structures/
    SnO2.POSCAR
    Sb2O3.POSCAR
    ZrO2.POSCAR
    O2.POSCAR
```

- `SnO2.POSCAR`: pristine unit cell for Host strucutre.
- Elemental `*.POSCAR`: bulk oxide reference structures for chemical potentials.

4.2.4 Example input.toml

```
[structure]
outdir = "random_structures"

[references]
reference_mode = "oxide"
host = "SnO2"
host_dir = "reference_structures/"
supercell = [ 5, 2, 1]
oxides_ref = [ "Sb2O3", "ZrO2"]
oxides_dir = "reference_structures/"
gas_ref = "O2"
gas_dir = "reference_structures/"
oxygen_mode = "O-rich"
muO_shift_ev = 0.0
fmax = 0.02
max_steps = 300
tf_threads = 1
omp_threads = 1
device = "cpu"
gpu_id = 0
backend = "m3gnet"
model = "default"
task = ""
optimizer = "bfgs"
skip_if_done = false

[doping]
mode = "explicit"
host_species = "Sn"
compositions = [
{ Sb = 5.0, Zr = 5.0 }
```

(continues on next page)

(continued from previous page)

```
]

[generate]
poscar_order = ["Zr","Sb","Sn","O"]
seed_base = 12345
clean_outdir = true

[scan]
backend = "m3gnet"
model = "default"
task = ""
poscar_in = "POSCAR"
topk = 10
symprec = 1e-3
max_enum = 10
n_workers = 4
chunksize = 10
anion_species = ["O"]
max_unique = 50000
skip_if_done = false
mode = "auto"
sample_budget = 5000
sample_batch_size = 64
sample_patience = 1000
sample_seed = 42
sample_max_saved = 10000
device = "cpu"

[relax]
backend = "m3gnet"
model = "default"
task = ""
optimizer = "bfgs"
fmax = 0.05
max_steps = 300
relax_mode = "atoms"
cell_filter = "frechet"
n_workers = 6
tf_threads = 1
omp_threads = 1
skip_if_done = false
skip_candidate_if_done = false
device = "cpu"
gpu_id = 0

[filter]
mode = "window"
window_meV = 50.0
max_candidates = 12
skip_if_done = false

[bandgap]
```

(continues on next page)

(continued from previous page)

```
skip_if_done = false
cutoff = 8.0
max_neighbors = 12
n_workers = 4
device = "cpu"

[formation]
skip_if_done = true
normalize = "per_dopant"

[database]
```

4.2.5 Outputs

- `reference_structures/reference_energies.json`
- `random_structures/<composition>/` - `ranking_scan.csv` - `ranking_relax.csv` - `ranking_relax_filtered.csv` - `bandgap_alignn_summary.csv` - `formation_energies.csv`
- `results_database.csv`

4.3 Multiple Oxide References and Relative Energies

This example evaluates Sb-doped SnO2 against three separate Sb oxide references. The workflow keeps **one candidate per row** and writes a separate column for all energies calculated from each oxide reference.

4.3.1 Configuration

Add every required binary oxide to the existing `[references].oxides_ref` list:

```
[references]
reference_mode = "oxide"
host = "SnO2"
host_dir = "reference_structures/"
supercell = [2, 2, 5]
oxides_dir = "reference_structures/"
oxides_ref = ["SbO2", "Sb2O3", "Sb2O5"]
gas_ref = "O2"
gas_dir = "reference_structures/"

[formation]
skip_if_done = false
normalize = "per_dopant"
relative_enabled = true
endpoint_x = "auto"
```

Both relative-energy parameters remain directly inside the existing `[formation]` section. No nested configuration section is required. `"auto"` identifies the pure oxide endmember and is recorded as `endpoint_x = 1.0` in the output metadata.

Place one POSCAR file for every oxide in `reference_structures/`:

```
reference_structures/Sb02.POSCAR
reference_structures/Sb203.POSCAR
reference_structures/Sb205.POSCAR
```

Then rebuild the cached reference energies before calculating formation energies:

```
dopingflow refs-build -c input.toml
dopingflow formation -c input.toml
dopingflow collect -c input.toml
```

4.3.2 Wide output format

For one Sb-doped candidate, the output remains one row. The oxide choice is encoded in the column suffix:

```
candidate,x_dopant,E_form_eV_total__Sb02,E_form_eV_total__Sb203,E_form_eV_total__Sb205,..
↪.
candidate_000,0.05,...,...,...,...
```

The following values are written for every oxide reference:

- E_form_eV_total__<reference>
- E_form_eV_per_atom__<reference>
- E_form_eV_per_cation__<reference>
- E_form_eV_per_dopant__<reference>
- E_mix_eV_total__<reference>
- E_mix_eV_per_atom__<reference>
- E_mix_eV_per_cation__<reference>
- E_mix_eV_per_dopant__<reference>
- E_form_rel_eV_per_cation__<reference>
- E_mix_rel_eV_per_cation__<reference>

4.3.3 Relative-energy definition

The oxide chemical potentials and atom-balanced mixing reaction already place each candidate on the host-oxide/dopant-oxide tie-line reference. Applying a second database-derived endpoint subtraction would double-correct the energy. Therefore the relative per-cation columns preserve the corresponding oxide-referenced formation and mixing values. The pure oxide endpoint energy, the composition-weighted co-doping correction, and the reference identity are written as separate provenance columns.

Collection and sequential merging preserve these formation-stage values. The legacy database endpoint calculation is used only when reading an older result database that has absolute reference-specific columns but no relative columns.

4.3.4 Co-doping

For co-doping, dopingflow evaluates all valid combinations of the listed binary oxide references. For example, Sb2O3 together with TiO2 is written as one suffix:

```
E_mix_eV_per_cation__Sb203__Ti02
```

The dopant order in these combined suffixes is alphabetical and deterministic.

4.4 Explicit Example — Multiple Target Compositions

4.4.1 Purpose

Generate several explicitly defined compositions in one run.

Each composition produces its own folder under `random_structures/`.

4.4.2 Run

```
dopingflow run-all -c input.toml
```

4.4.3 Example input.toml

```
[structure]
outdir = "random_structures"

[references]
reference_mode = "metal"
host = "SnO2"
host_dir = "reference_structures/"
supercell = [ 5, 2, 1]
metals_ref = [ "Ti","Zr","Nb","Sb","Sn"]
oxides_dir = "reference_structures/"
fmax = 0.02
max_steps = 300
tf_threads = 1
omp_threads = 1
device = "cpu"
gpu_id = 0
backend = "m3gnet"
model = "default"
task = ""
optimizer = "bfgs"
skip_if_done = false

[generate]
poscar_order = ["Ti","Zr","Nb","Sb","Sn","O"]
seed_base = 2026
clean_outdir = true

[doping]
mode = "explicit"
host_species = "Sn"
compositions = [
  { Sb = 5.0, Ti = 5.0 },
  { Sb = 5.0, Zr = 5.0 },
  { Sb = 10.0, Nb = 5.0 }
]

# Other sections identical to previous example
```

4.4.4 Result

Each composition gets its own directory:

```
random_structures/
    Sb5_Ti5/
    Sb5_Zr5/
    Sb10_Nb5/
```

Each folder contains full scan, relax, filter, bandgap and formation results.

4.5 Enumerate Example — Systematic Dopant Screening

4.5.1 Purpose

Screen multiple dopants and concentrations automatically.

Use this mode when: - You want combinatorial exploration. - You define allowed dopants and doping levels. - You limit the maximum number of dopants per structure.

4.5.2 Note

The current implementation supports **up to 3 dopants per structure** via:

```
max_dopants_total = 3
```

4.5.3 Run

```
dopingflow run-all -c input.toml
```

4.5.4 Example input.toml

```
[structure]
outdir = "random_structures"

[references]
reference_mode = "metal"
host = "SnO2"
host_dir = "reference_structures/"
supercell = [ 5, 2, 1]
metals_ref = [ "Ti","Zr","Nb","Sb","Sn"]
oxides_dir = "reference_structures/"
fmax = 0.02
max_steps = 300
tf_threads = 1
omp_threads = 1
device = "cpu"
gpu_id = 0
backend = "m3gnet"
model = "default"
task = ""
optimizer = "bfgs"
skip_if_done = false
```

(continues on next page)

(continued from previous page)

```
[generate]
poscar_order = ["Ti","Zr","Nb","Sb","Sn","O"]
seed_base = 12345
clean_outdir = true

[doping]
mode = "enumerate"
host_species = "Sn"
dopants = ["Sb", "Ti", "Zr", "Nb"]
must_include = ["Sb"]
max_dopants_total = 3
allowed_totals = [5.0, 10.0, 15.0]
levels = [5.0, 10.0, 15.0]

# Other sections identical to previous example
```

4.5.5 Result

The workflow automatically generates composition folders like:

```
random_structures/
    Sb5/
    Sb5_Ti5/
    Sb5_Ti5_Zr5/
    Sb10_Nb5/
    ...
```

Each is processed independently through all stages.

4.6 Smoke Test — Minimal Fast Run

4.6.1 Purpose

Quickly test installation and pipeline functionality with reduced computational cost.

This example: - Uses smaller supercell - Uses small topk - Uses top-N filtering - Can stop early

Run only steps 00–04:

```
dopingflow run-all -c input.toml --start 0 --stop 4
```

4.6.2 Example input.toml

```
[structure]
outdir = "random_structures"

[references]
reference_mode = "metal"
host = "SnO2"
host_dir = "reference_structures/"
supercell = [ 5, 2, 1]
```

(continues on next page)

(continued from previous page)

```
metals_ref = [ "Ti","Zr","Nb","Sb","Sn"]
oxides_dir = "reference_structures/"
fmax = 0.02
max_steps = 300
tf_threads = 1
omp_threads = 1
device = "cpu"
gpu_id = 0
backend = "m3gnet"
model = "default"
task = ""
optimizer = "bfgs"
skip_if_done = false

[generate]
poscar_order = ["Ti","Zr","Nb","Sb","Sn","O"]
seed_base = 2026
clean_outdir = true

[scan]
backend = "m3gnet"
model = "default"
task = ""
poscar_in = "POSCAR"
topk = 10
symprec = 1e-3
max_enum = 10
n_workers = 4
chunksize = 10
anion_species = ["O"]
max_unique = 50000
skip_if_done = false
mode = "auto"
sample_budget = 5000
sample_batch_size = 64
sample_patience = 1000
sample_seed = 42
sample_max_saved = 10000
device = "cpu"

[doping]
mode = "explicit"
host_species = "Sn"
compositions = [
{ Sb = 5.0, Ti = 5.0 },
{ Sb = 5.0, Zr = 5.0 },
{ Sb = 10.0, Nb = 5.0 }
]

[relax]
backend = "m3gnet"
model = "default"
```

(continues on next page)

(continued from previous page)

```
task = ""
optimizer = "bfgs"
fmax = 0.05
max_steps = 300
relax_mode = "atoms"
cell_filter = "frechet"
n_workers = 6
tf_threads = 1
omp_threads = 1
skip_if_done = false
skip_candidate_if_done = false
device = "cpu"
gpu_id = 0

[filter]
mode = "window"
window_meV = 50.0
max_candidates = 12
skip_if_done = false

[bandgap]
skip_if_done = false
cutoff = 8.0
max_neighbors = 12
n_workers = 4
device = "cpu"

[formation]
skip_if_done = true
normalize = "per_dopant"

[database]
```

4.6.3 Expected Result

You should obtain:

```
random_structures/<composition>/
    ranking_scan.csv
    ranking_relax.csv
    ranking_relax_filtered.csv
```

This confirms: - generation works - scan works - relaxation works - filtering works

4.7 Sequential Example — Gradual Sb Doping

4.7.1 Purpose

This example performs **gradual sequential Sb doping** of `Sn02`.

Use this mode when:

- You want to increase the dopant concentration step by step.

- You want each composition to start from the lowest-energy relaxed structure of the previous composition.
- You want to improve structural optimization along a doping path.
- You want to recompute formation and mixing energies later using different oxide references without regenerating or relaxing structures.

4.7.2 Workflow

Run the sequential workflow:

```
dopingflow sequential-run -c input.toml
```

In `mode = "full"`, each composition step runs:

```
generate -> scan -> relax -> filter -> optional bandgap -> formation -> collect
```

After each step, the lowest-energy relaxed structure is copied to:

```
sequential_structures/step_xxx_<composition>/best_relaxed/POSCAR
```

and used as the starting structure for the next composition.

4.7.3 Required Files

The working directory must contain:

```
input.toml
reference_structures/
    SnO2.POSCAR
    Sb2O5.POSCAR
    O2.POSCAR
```

- `SnO2.POSCAR`: pristine host oxide structure.
- `Sb2O5.POSCAR`: dopant oxide reference structure.
- `O2.POSCAR`: oxygen gas reference structure.

4.7.4 Example input.toml

```
[structure]
outdir = "Sb_sequential_SnO2_mace"

[references]
reference_mode = "oxide"
skip_if_done = false
fmax = 0.02
max_steps = 300
tf_threads = 1
omp_threads = 1
device = "cpu"
gpu_id = 0
backend = "mace"
model = "small"
task = ""
```

(continues on next page)

(continued from previous page)

```
optimizer = "bfgs"
host = "SnO2"
host_dir = "reference_structures/"
supercell = [2, 2, 5]
metal_ref = ["Sn", "Sb"]
metals_dir = "reference_structures/"
oxides_ref = ["Sb2O5"]
oxides_dir = "reference_structures/"
gas_ref = "O2"
gas_dir = "reference_structures/"
oxygen_mode = "O-rich"
muO_shift_ev = 0.0

[generate]
poscar_order = ["Sb", "Sn", "O"]
seed_base = 12345

[sequential]
outdir = "sequential_structures"
mode = "full"

[doping]
mode = "enumerate"
host_species = "Sn"
must_include = ["Sb"]
dopants = ["Sb"]
max_dopants_total = 1
allowed_totals = [2.5, 5.0, 7.5, 10.0]
levels = [2.5, 5.0, 7.5, 10.0]

[scan]
backend = "mace"
model = "small"
task = ""
poscar_in = "POSCAR"
topk = 20
symprec = 0.001
n_workers = 8
chunksize = 10
max_enum = 10000
max_unique = 5000
anion_species = ["O"]
skip_if_done = false
mode = "auto"
sample_budget = 10000
sample_batch_size = 20
sample_patience = 60
sample_seed = 42
sample_max_saved = 1000
device = "cpu"
gpu_id = 0
```

(continues on next page)

(continued from previous page)

```
[relax]
backend = "mace"
model = "small"
task = ""
relax_mode = "full"
cell_filter = "frechet"
optimizer = "bfgs"
fmax = 0.05
max_steps = 300
n_workers = 4
tf_threads = 1
omp_threads = 1
skip_if_done = false
skip_candidate_if_done = false
device = "cpu"
gpu_id = 0

[filter]
mode = "window"
window_meV = 50.0
max_candidates = 12
skip_if_done = false

[bandgap]
enabled = false
skip_if_done = false
cutoff = 8.0
max_neighbors = 12
n_workers = 4
device = "cpu"
gpu_id = 0
batch_size = 32

[formation]
skip_if_done = false
normalize = "total"

[database]
skip_if_done = false
```

4.7.5 Recomputing Energies with a Different Reference

After the full sequential workflow has finished, the same relaxed structures can be reused to recompute formation and mixing energies with a different oxide reference.

For example, change:

```
oxides_ref = ["Sb2O5"]
```

to another reference, then rebuild references:

```
dopingflow refs-build -c input.toml
```

Then set:

```
[sequential]
mode = "recompute_energies"
```

and rerun:

```
dopingflow sequential-run -c input.toml
```

This skips:

```
generate -> scan -> relax -> filter -> bandgap
```

and reruns only:

```
formation -> collect
```

for the existing sequential structures.

4.7.6 Outputs

Each sequential step writes its own folder:

```
sequential_structures/
    step_001_Sb2p5/
    step_002_Sb5/
    step_003_Sb7p5/
    step_004_Sb10/
```

Each step contains:

- `input_step.json`
- `random_structures/<composition>/`
- `best_relaxed/POSCAR`
- `results_database.csv`
- `sequential_step_summary.json`

The final merged database is written to the project root:

```
results_database.csv
```

4.8 Vacancy workflow example

The complete example is in `examples/vacancies`. It assumes the normal bulk workflow has already produced selected relaxed candidates.

```
dopingflow vacancies -c examples/vacancies/input.toml
```

The example uses one flat `[vacancies]` table and a single MACE calculator for screening and relaxation. Adjust `device` and `model` to match the installed environment.

CHAPTER

FIVE

API REFERENCE

5.1 dopingflow

5.1.1 dopingflow package

Submodules

dopingflow.bandgap module

class dopingflow.bandgap.**BandgapConfig**(*outdir: 'Path', skip_if_done: 'bool', cutoff: 'float', max_neighbors: 'int', n_workers: 'int', device: 'str', gpu_id: 'int', batch_size: 'int'*)

Bases: object

Parameters

- **outdir** (*Path*)
- **skip_if_done** (*bool*)
- **cutoff** (*float*)
- **max_neighbors** (*int*)
- **n_workers** (*int*)
- **device** (*str*)
- **gpu_id** (*int*)
- **batch_size** (*int*)

outdir: Path

skip_if_done: bool

cutoff: float

max_neighbors: int

n_workers: int

device: str

gpu_id: int

batch_size: int

dopingflow.bandgap.**run_bandgap**(*raw_cfg*, *root*, ** (Keyword-only parameters separator (PEP 3102))*, *config_path=None*)

Step 05: Predict bandgap (ALIGNN local model) for relaxed candidates.

Selection:

1) If selected_candidates.txt exists in a composition folder -> use it
2) Else fallback to candidate_*/02_relax/POSCAR

Outputs per composition folder:

- bandgap_alignn_summary.csv
- candidate_*/03_band/meta.json

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type
None

dopingflow.bandgap.**run_bandgap_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
None

dopingflow.cli module

dopingflow.cli.**refs_build_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 00: Build/cache reference energies.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**generate_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 01: Generate random doped structures.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**scan_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 02: Symmetry-unique scan + M3GNet single-point energies (top-k).

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**relax_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 03: Relax scanned candidates with M3GNet Relaxer.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**filter_cmd**(*config=<typer.models.OptionInfo object>*, *only=<typer.models.OptionInfo object>*, *force=<typer.models.OptionInfo object>*, *window_meV=<typer.models.OptionInfo object>*, *topn=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 04: Filter relaxed candidates (window or top-N).

Parameters

- **config** (*Path*)
- **only** (*str | None*)
- **force** (*bool*)
- **window_meV** (*float | None*)
- **topn** (*int | None*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**bandgap_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 05: Predict bandgap for filtered relaxed candidates (ALIGNN).

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**formation_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 06: Compute formation energies using cached references.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**collect_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 07: Collect selected candidates into one CSV database.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**alloy_hull_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 08: Build restricted one-dimensional alloy convex hull.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**phase_diagram_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Step 09: Compute full phase-diagram energy above hull.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**vacancies_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Determine vacancy counts, enumerate, ML-screen, select top-k, and relax.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**run_all_cmd**(*config=<typer.models.OptionInfo object>*, *start=<typer.models.OptionInfo object>*, *stop=<typer.models.OptionInfo object>*, *only=<typer.models.OptionInfo object>*, *dry_run=<typer.models.OptionInfo object>*, *filter_only=<typer.models.OptionInfo object>*, *force=<typer.models.OptionInfo object>*, *window_meV=<typer.models.OptionInfo object>*, *topn=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Run the full pipeline in order, with optional step selection.

Step keys:
refs -> generate -> scan -> relax -> filter -> bandgap -> formation -> collect -> alloy-hull -> phase-diagram -> vacancies

Parameters

- **config** (*Path*)
- **start** (*str*)
- **stop** (*str*)
- **only** (*str | None*)
- **dry_run** (*bool*)
- **filter_only** (*str | None*)
- **force** (*bool*)
- **window_meV** (*float | None*)
- **topn** (*int | None*)
- **verbose** (*bool*)

Return type
None

dopingflow.cli.**sequential_run_cmd**(*config=<typer.models.OptionInfo object>*, *verbose=<typer.models.OptionInfo object>*)

Run gradual sequential doping workflow.

Parameters

- **config** (*Path*)
- **verbose** (*bool*)

Return type
None

dopingflow.collect module

class dopingflow.collect.**DBConfig**(*outdir: 'Path'*, *skip_if_done: 'bool'*)

Bases: object

Parameters

- **outdir** (*Path*)
- **skip_if_done** (*bool*)

outdir: Path

skip_if_done: bool

dopingflow.collect.**read_json**(*path*)

Parameters
path (*Path*)

Return type
dict | None

dopingflow.collect.**safe_get**(*d*, **keys*, *default=None*)

Parameters
d (*dict | None*)

dopingflow.collect.**read_selected_txt**(*path*)

Parameters
path (*Path*)

Return type
List[str]

dopingflow.collect.**read_filtered_table**(*path*)

Parameters
path (*Path*)

Return type
Dict[str, *Dict*[str, *Any*]]

dopingflow.collect.**read_scan_ranking**(*path*)

Parameters
path (*Path*)

Return type
Dict[str, *Dict*[str, *Any*]]

dopingflow.collect.**read_bandgap_summary**(*path*)

Parameters
path (*Path*)

Return type
Dict[str, *Dict*[str, *Any*]]

dopingflow.collect.**read_formation_csv**(*path*)

formation_energies.csv written by formation.py.

This remains a fallback. The preferred source is candidate_*/04_formation/meta.json, because it contains the full `reference_results` block for multi-oxide runs.

Parameters
path (*Path*)

Return type
Dict[str, *Dict*[str, *Any*]]

dopingflow.collect.**read_formation_meta**(*path*)

candidate_*/04_formation/meta.json written by formation.py.

Backward compatibility:

- keeps the old primary/top-level fields
- additionally exposes wide multi-oxide fields from reference_results

Parameters
path (*Path*)

Return type
Dict[str, *Any*]

dopingflow.collect.**run_collect**(*raw_cfg*, *root*, *, *config_path=None*)

Step 07: Collect results into ONE flat CSV database (results_database.csv), only for filtered/selected candidates.

This collector preserves the historical primary columns and adds wide columns for every oxide-reference scenario found in formation.py's `reference_results`.

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type
Path

dopingflow.collect.**run_collect_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
Path

dopingflow.filtering module

class dopingflow.filtering.**FilterConfig**(*outdir: 'Path'*, *mode: 'str'*, *window_meV: 'float'*, *max_candidates: 'int'*, *skip_if_done: 'bool'*)

Bases: `object`

Parameters

- **outdir** (*Path*)
- **mode** (*str*)
- **window_meV** (*float*)
- **max_candidates** (*int*)
- **skip_if_done** (*bool*)

outdir: Path

mode: str

window_meV: float

max_candidates: int

skip_if_done: bool

dopingflow.filtering.**run_filtering**(*raw_cfg*, *root*, *, *only=None*, *force=False*, *window_meV=None*, *topn=None*)

Step 04: filter relaxed candidates after Step 03.

For each composition folder in [structure].outdir:
reads: ranking_relax.csv writes: ranking_relax_filtered.csv, selected_candidates.txt

Filtering:

- [filter].mode="window": keep candidates within window_meV above Emin
- [filter].mode="topn": keep lowest-energy max_candidates

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **only** (*str | None*)
- **force** (*bool*)
- **window_meV** (*float | None*)
- **topn** (*int | None*)

Return type
None

dopingflow.filtering.**run_filtering_from_toml**(*config_path*, *, *only=None*, *force=False*, *window_meV=None*, *topn=None*)

Parameters

- **config_path** (*Path*)
- **only** (*str | None*)
- **force** (*bool*)
- **window_meV** (*float | None*)
- **topn** (*int | None*)

Return type
None

dopingflow.formation module

class dopingflow.formation.**FormationConfig**(*outdir: 'Path'*, *host_species: 'str'*, *anion_species: 'List[str]'*, *skip_if_done: 'bool'*, *normalize: 'str'*, *relative_enabled: 'bool'*, *relative_endpoint_x: 'float | None'*)

Bases: object

Parameters

- **outdir** (*Path*)
- **host_species** (*str*)

- **anion_species** (*List[str]*)
- **skip_if_done** (*bool*)
- **normalize** (*str*)
- **relative_enabled** (*bool*)
- **relative_endpoint_x** (*float | None*)

outdir: Path

host_species: str

anion_species: List[str]

skip_if_done: bool

normalize: str

relative_enabled: bool

relative_endpoint_x: float | None

class dopingflow.formation.**CandidateRecord**(*folder: 'Path'*, *candidate_dir: 'Path'*, *candidate: 'str'*, *E_doped: 'float'*, *counts: 'Dict[str, int]'*, *dopant_counts: 'Dict[str, int]'*, *x_dopant: 'float'*, *reference_results: 'Dict[str, Dict[str, Any]]'*)

Bases: object

Parameters

- **folder** (*Path*)
- **candidate_dir** (*Path*)
- **candidate** (*str*)
- **E_doped** (*float*)
- **counts** (*Dict[str, int]*)
- **dopant_counts** (*Dict[str, int]*)
- **x_dopant** (*float*)
- **reference_results** (*Dict[str, Dict[str, Any]]*)

folder: Path

candidate_dir: Path

candidate: str

E_doped: float

counts: Dict[str, int]

dopant_counts: Dict[str, int]

x_dopant: float

reference_results: Dict[str, Dict[str, Any]]

dopingflow.formation.**run_formation**(*raw_cfg*, *root*, *, *config_path=None*)

Compute formation and oxide-reference mixing energies for relaxed candidates.

In oxide mode every binary oxide reference matching a dopant is evaluated. Results are written in wide format: one candidate row and one set of columns per reference scenario (for example __Sb02 and __Sb205).

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type
None

dopingflow.formation.**run_formation_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
None

dopingflow.generate module

class dopingflow.generate.**GenerateConfig**(*base_poscar: 'str | None'*, *incremental: 'bool'*, *outdir: 'str'*, *poscar_order: 'List[str]'*, *seed_base: 'int'*, *clean_outdir: 'bool'*, *mode: 'str'*, *host_species: 'str'*, *compositions: 'List[Dict[str, float]]'*, *dopants: 'List[str]'*, *must_include: 'List[str]'*, *max_dopants_total: 'int'*, *allowed_totals: 'List[float]'*, *levels: 'List[float]'*)

Bases: object

Parameters

- **base_poscar** (*str | None*)
- **incremental** (*bool*)
- **outdir** (*str*)
- **poscar_order** (*List[str]*)
- **seed_base** (*int*)
- **clean_outdir** (*bool*)
- **mode** (*str*)
- **host_species** (*str*)
- **compositions** (*List[Dict[str, float]]*)
- **dopants** (*List[str]*)
- **must_include** (*List[str]*)
- **max_dopants_total** (*int*)
- **allowed_totals** (*List[float]*)
- **levels** (*List[float]*)

base_poscar: str | None

incremental: bool

outdir: str

poscar_order: List[str]

seed_base: int

clean_outdir: bool

mode: str

host_species: str

compositions: List[Dict[str, float]]

dopants: List[str]

must_include: List[str]

max_dopants_total: int

allowed_totals: List[float]

levels: List[float]

dopingflow.generate.**validate_composition_minimal**(*requested_pct*)

Parameters
requested_pct (*Dict[str, float]*)

Return type
None

dopingflow.generate.**normalize_to_counts_and_effective**(*n_host*, *requested_pct*)

Convert requested dopant percentages (relative to host sites) into integer dopant counts by rounding to the nearest integer number of substitutions.

Parameters

- **n_host** (*int*)
- **requested_pct** (*Dict[str, float]*)

Return type
tuple[*Dict*[str, int], *Dict*[str, float], *List*[str], float, float]

dopingflow.generate.**normalize_incremental_counts**(*base*, *host_species*, *requested_pct*)

For sequential doping: - count existing dopants in the base structure - compute target dopant counts - return only the additional dopants to add

Parameters

- **base** (*Structure*)
- **host_species** (*str*)
- **requested_pct** (*Dict[str, float]*)

Return type
tuple[*Dict*[str, int], *Dict*[str, int], *Dict*[str, int], *Dict*[str, float], *List*[str], int]

dopingflow.generate.**build_incremental_structure_from_counts**(*base*, *host_species*, *additional_counts*, *seed*)

Parameters

- **base** (*Structure*)
- **host_species** (*str*)
- **additional_counts** (*Dict[str, int]*)
- **seed** (*int*)

Return type
Structure

dopingflow.generate.**composition_tag**(*effective_pct*, *must_first=None*)

Parameters

- **effective_pct** (*Dict[str, float]*)
- **must_first** (*List[str] | None*)

Return type
str

dopingflow.generate.**stable_seed_from_tag**(*tag*, *seed_base*)

Parameters

- **tag** (*str*)
- **seed_base** (*int*)

Return type
int

dopingflow.generate.**build_structure_from_counts**(*pristine*, *host_species*, *dopant_counts*, *seed*)

Parameters

- **pristine** (*Structure*)
- **host_species** (*str*)
- **dopant_counts** (*Dict[str, int]*)
- **seed** (*int*)

Return type
Structure

dopingflow.generate.**reorder_structure_by_species**(*s*, *order*)

Parameters

- **s** (*Structure*)
- **order** (*List[str]*)

Return type
Structure

dopingflow.generate.**enumerate_compositions**(*dopants*, *must_include*, *max_dopants_total*, *allowed_totals*, *levels*)

Parameters

- **dopants** (*List[str]*)
- **must_include** (*List[str]*)
- **max_dopants_total** (*int*)
- **allowed_totals** (*List[float]*)
- **levels** (*List[float]*)

Return type
List[*Dict*[str, float]]

dopingflow.generate.**run_generate**(*raw_cfg*, *root*, *, *config_path=None*)

Generate one random doped POSCAR per composition.

Requires:

- refs-build completed
- reference_structures/reference_energies.json exists
- relaxed host supercell POSCAR exists

Output:
<outdir>/<tag>/POSCAR <outdir>/<tag>/metadata.json

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type
Path

dopingflow.generate.**run_generate_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
Path

dopingflow.hardware module

class dopingflow.hardware.**HardwareConfig**(*device: 'DeviceMode' = 'auto'*, *gpu_id: 'int' = 0*, *allow_gpu_batching: 'bool' = True*, *relax_batch_size: 'int' = 8*, *bandgap_batch_size: 'int' = 32*)

Bases: object

Parameters

- **device** (*Literal['auto', 'cpu', 'cuda']*)
- **gpu_id** (*int*)
- **allow_gpu_batching** (*bool*)

- **relax_batch_size** (*int*)
- **bandgap_batch_size** (*int*)

device: `Literal['auto', 'cpu', 'cuda'] = 'auto'`

gpu_id: `int = 0`

allow_gpu_batching: `bool = True`

relax_batch_size: `int = 8`

bandgap_batch_size: `int = 32`

dopingflow.hardware.**resolve_torch_device**(*mode='auto'*, *gpu_id=0*)

Parameters

- **mode** (*str*)
- **gpu_id** (*int*)

Return type
str

dopingflow.hardware.**configure_tensorflow**(*mode='auto'*, *gpu_id=0*)

Parameters

- **mode** (*str*)
- **gpu_id** (*int*)

Return type
str

dopingflow.hardware.**parse_hardware_config**(*raw_cfg*)

Parameters
raw_cfg (*dict*)

Return type
HardwareConfig

dopingflow.logging module

dopingflow.logging.**setup_logging**(*root*, *, *verbose=False*)

Configure logging for dopingflow.

- Console output
- Per-run log file
- Noise suppression

Parameters

- **root** (*Path*)
- **verbose** (*bool*)

Return type
Path

dopingflow.ml_backends module

dopingflow.ml_backends.**get_mace_model_choices**()

Return the aliases supported by the installed `mace_mp` calculator.

MACE owns this catalogue and may add aliases between releases. Keeping the installed package authoritative prevents dopingflow from rejecting a model that the user's MACE version already supports. The fallback keeps the GUI and validation useful when MACE is an optional, not-yet-installed backend.

Return type
Tuple[str, ...]

dopingflow.ml_backends.**set_default_runtime_env**(*, *tf_threads=1*, *omp_threads=1*)

Conservative defaults to keep CPU/TensorFlow noise low. Safe to call repeatedly.

Parameters

- **tf_threads** (*int*)
- **omp_threads** (*int*)

Return type
None

dopingflow.ml_backends.**normalize_backend_config**(*, *backend*, *model*, *task*, *section_name*)

Normalize + validate backend/model/task choices for any stage. Returns: (backend, model, task)

Parameters

- **backend** (*str*)
- **model** (*str*)
- **task** (*str*)
- **section_name** (*str*)

Return type
Tuple[str, str, str]

dopingflow.ml_backends.**check_backend_dependency**(*backend*, *, *stage_name*)

Fail early with clear stage-specific import errors.

Parameters

- **backend** (*str*)
- **stage_name** (*str*)

Return type
None

dopingflow.ml_backends.**prepare_backend_runtime**(*, *backend*, *device*, *gpu_id*, *tf_threads=1*, *omp_threads=1*)

Configure backend runtime environment in the current process. Call this before constructing the calculator.

Parameters

- **backend** (*str*)
- **device** (*str*)
- **gpu_id** (*int*)
- **tf_threads** (*int*)

- **omp_threads** (*int*)

Return type
None

dopingflow.ml_backends.**build_ase_calculator**(*, *backend*, *model*, *task*, *device*)

Return an ASE-compatible calculator for the selected backend.

Parameters

- **backend** (*str*)
- **model** (*str*)
- **task** (*str*)
- **device** (*str*)

dopingflow.ml_relaxation module

dopingflow.ml_relaxation.**get_optimizer_class**(*name*)

Parameters
name (*str*)

dopingflow.ml_relaxation.**final_fmax**(*forces*)

Parameters
forces (*ndarray*)

Return type
float

dopingflow.ml_relaxation.**structure_energy_with_calculator**(*struct*, *calculator*)

Parameters
struct (*Structure*)

Return type
float

dopingflow.ml_relaxation.**build_relaxation_target**(*atoms*, *relax_mode*, *cell_filter*)

Parameters

- **relax_mode** (*str*)
- **cell_filter** (*str*)

dopingflow.ml_relaxation.**relax_structure_with_calculator**(*struct*, *, *calculator*, *optimizer_name*, *fmax*, *max_steps*, *relax_mode='atoms'*, *cell_filter='frechet'*)

Parameters

- **struct** (*Structure*)
- **optimizer_name** (*str*)
- **fmax** (*float*)
- **max_steps** (*int*)
- **relax_mode** (*str*)
- **cell_filter** (*str*)

Return type
Tuple[*Structure*, float, int, float, bool]

dopingflow.refs module

class dopingflow.refs.**RefConfig**(*reference_mode: 'str'*, *skip_if_done: 'bool'*, *fmax: 'float'*, *max_steps: 'int'*, *tf_threads: 'int'*, *omp_threads: 'int'*, *device: 'str'*, *gpu_id: 'int'*, *backend: 'str'*, *model: 'str'*, *task: 'str'*, *optimizer: 'str'*, *host: 'str'*, *host_dir: 'Path'*, *supercell: 'tuple[int, int, int]'*, *metal_ref: 'List[str]'*, *metals_dir: 'Path'*, *oxides_ref: 'List[str]'*, *oxides_dir: 'Path'*, *gas_ref: 'str'*, *gas_dir: 'Path'*, *oxygen_mode: 'str'*, *muO_shift_ev: 'float'*)

Bases: `object`

Parameters

- **reference_mode** (*str*)
- **skip_if_done** (*bool*)
- **fmax** (*float*)
- **max_steps** (*int*)
- **tf_threads** (*int*)
- **omp_threads** (*int*)
- **device** (*str*)
- **gpu_id** (*int*)
- **backend** (*str*)
- **model** (*str*)
- **task** (*str*)
- **optimizer** (*str*)
- **host** (*str*)
- **host_dir** (*Path*)
- **supercell** (*tuple[int, int, int]*)
- **metal_ref** (*List[str]*)
- **metals_dir** (*Path*)
- **oxides_ref** (*List[str]*)
- **oxides_dir** (*Path*)
- **gas_ref** (*str*)
- **gas_dir** (*Path*)
- **oxygen_mode** (*str*)
- **muO_shift_ev** (*float*)

reference_mode: str

skip_if_done: bool

fmax: float

max_steps: int

tf_threads: int

omp_threads: int

device: str

gpu_id: int

backend: str

model: str

task: str

optimizer: str

host: str

host_dir: Path

supercell: tuple[int, int, int]

metal_ref: List[str]

metals_dir: Path

oxides_ref: List[str]

oxides_dir: Path

gas_ref: str

gas_dir: Path

oxygen_mode: str

mu0_shift_ev: float

dopingflow.refs.**run_refs_build**(*raw_cfg*, *root*, *, *config_path=None*)

Build/cache relaxed reference energies needed for formation energy calculations.

Outputs:

- reference_structures/reference_energies.json
- reference_structures/relaxed/host_unit_relaxed.POSCAR
- reference_structures/relaxed/host_supercell_<a>x<b>x<c>_relaxed.POSCAR
- reference_structures/relaxed/refs/<name>_relaxed.POSCAR

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type

Path

dopingflow.refs.**run_refs_build_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
Path

dopingflow.relax module

class dopingflow.relax.**RelaxConfig**(*fmax: 'float', max_steps: 'int', n_workers: 'int', tf_threads: 'int', omp_threads: 'int', order: 'List[str]', outdir: 'Path', skip_if_done: 'bool', skip_candidate_if_done: 'bool', device: 'str', gpu_id: 'int', relax_mode: 'str', cell_filter: 'str', backend: 'str', model: 'str', task: 'str', optimizer: 'str'*)

Bases: object

Parameters

- **fmax** (*float*)
- **max_steps** (*int*)
- **n_workers** (*int*)
- **tf_threads** (*int*)
- **omp_threads** (*int*)
- **order** (*List[str]*)
- **outdir** (*Path*)
- **skip_if_done** (*bool*)
- **skip_candidate_if_done** (*bool*)
- **device** (*str*)
- **gpu_id** (*int*)
- **relax_mode** (*str*)
- **cell_filter** (*str*)
- **backend** (*str*)
- **model** (*str*)
- **task** (*str*)
- **optimizer** (*str*)

fmax: float

max_steps: int

n_workers: int

tf_threads: int

omp_threads: int

order: List[str]

outdir: Path

skip_if_done: bool

skip_candidate_if_done: bool

device: str

gpu_id: int

relax_mode: str

cell_filter: str

backend: str

model: str

task: str

optimizer: str

dopingflow.relax.**run_relax**(*raw_cfg*, *root*, ***, *config_path=None*)

Step 03: Relax candidates produced by Step 02 using a unified ASE backend layer.

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type
None

dopingflow.relax.**run_relax_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
None

dopingflow.scan module

class dopingflow.scan.**ScanConfig**(*poscar_in: 'str'*, *topk: 'int'*, *symprec: 'float'*, *max_enum: 'int'*, *n_workers: 'int'*, *chunksize: 'int'*, *order: 'List[str]'*, *anion_species: 'List[str]'*, *host_species: 'str'*, *max_unique: 'int'*, *skip_if_done: 'bool'*, *device: 'str'*, *gpu_id: 'int'*, *backend: 'str'*, *model: 'str'*, *task: 'str'*, *mode: 'str'*, *sample_budget: 'int'*, *sample_batch_size: 'int'*, *sample_patience: 'int'*, *sample_seed: 'int'*, *sample_max_saved: 'int'*)

Bases: object

Parameters

- **poscar_in** (*str*)
- **topk** (*int*)
- **symprec** (*float*)
- **max_enum** (*int*)

- **n_workers** (*int*)
- **chunksize** (*int*)
- **order** (*List[str]*)
- **anion_species** (*List[str]*)
- **host_species** (*str*)
- **max_unique** (*int*)
- **skip_if_done** (*bool*)
- **device** (*str*)
- **gpu_id** (*int*)
- **backend** (*str*)
- **model** (*str*)
- **task** (*str*)
- **mode** (*str*)
- **sample_budget** (*int*)
- **sample_batch_size** (*int*)
- **sample_patience** (*int*)
- **sample_seed** (*int*)
- **sample_max_saved** (*int*)

poscar_in: str

topk: int

symprec: float

max_enum: int

n_workers: int

chunksize: int

order: List[str]

anion_species: List[str]

host_species: str

max_unique: int

skip_if_done: bool

device: str

gpu_id: int

backend: str

model: str

task: str

mode: str

sample_budget: int

sample_batch_size: int

sample_patience: int

sample_seed: int

sample_max_saved: int

dopingflow.scan.**run_scan**(*raw_cfg*, *root*, *, *config_path=None*)

Step 02:

- enumerate / sample symmetry-unique dopant arrangements
- evaluate single-point energies using selected ML backend via ASE calculator
- keep top-k lowest energies

Parameters

- **raw_cfg** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type
None

dopingflow.scan.**run_scan_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
None

dopingflow.surface module

dopingflow.surface.**run_surface_from_toml**(*config_path*)

Load TOML config and run the surface generation stage.

Parameters
config_path (*Path*)

Return type
None

dopingflow.surface.**run_surface**(*config*)

Main entry point for surface generation (+ optional surface relaxation).

Parameters
config (*Dict[str, Any]*)

Return type
None

dopingflow.utils.io module

dopingflow.vacancies module

class dopingflow.vacancies.**VacancyConfig**(*outdir: 'Path', enabled: 'bool', parent_source: 'str', parent_directory: 'Path | None', include_parent_reference: 'bool', skip_if_done: 'bool', resume: 'bool', count_mode: 'str', host_species: 'str', host_oxidation_state: 'int', vacancy_species: 'str', vacancy_compensation_charge: 'int', oxidation_states: 'dict[str, tuple[int, ...]]', extra_vacancies: 'int', max_vacancies_cap: 'int', symprec: 'float', angle_tolerance: 'float', mapping_tolerance: 'float', enumeration_mode: 'str', max_exact_raw_configs: 'int', max_exact_unique_configs: 'int', sample_budget: 'int', sample_batch_size: 'int', sample_patience: 'int', sample_seed: 'int', sample_max_saved: 'int', minimum_vacancy_distance: 'float', backend: 'str', model: 'str', task: 'str', device: 'str', gpu_id: 'int', n_workers: 'int', tf_threads: 'int', omp_threads: 'int', chunksize: 'int', topk_per_vacancy_count: 'int', energy_normalization: 'str', optimizer: 'str', fmax: 'float', max_steps: 'int', relax_mode: 'str', cell_filter: 'str'*)

Bases: `object`

Parameters

- **outdir** (*Path*)
- **enabled** (*bool*)
- **parent_source** (*str*)
- **parent_directory** (*Path | None*)
- **include_parent_reference** (*bool*)
- **skip_if_done** (*bool*)
- **resume** (*bool*)
- **count_mode** (*str*)
- **host_species** (*str*)
- **host_oxidation_state** (*int*)
- **vacancy_species** (*str*)
- **vacancy_compensation_charge** (*int*)
- **oxidation_states** (*dict[str, tuple[int, ...]]*)
- **extra_vacancies** (*int*)
- **max_vacancies_cap** (*int*)
- **symprec** (*float*)
- **angle_tolerance** (*float*)
- **mapping_tolerance** (*float*)
- **enumeration_mode** (*str*)
- **max_exact_raw_configs** (*int*)

- **max_exact_unique_configs** (*int*)
- **sample_budget** (*int*)
- **sample_batch_size** (*int*)
- **sample_patience** (*int*)
- **sample_seed** (*int*)
- **sample_max_saved** (*int*)
- **minimum_vacancy_distance** (*float*)
- **backend** (*str*)
- **model** (*str*)
- **task** (*str*)
- **device** (*str*)
- **gpu_id** (*int*)
- **n_workers** (*int*)
- **tf_threads** (*int*)
- **omp_threads** (*int*)
- **chunksize** (*int*)
- **topk_per_vacancy_count** (*int*)
- **energy_normalization** (*str*)
- **optimizer** (*str*)
- **fmax** (*float*)
- **max_steps** (*int*)
- **relax_mode** (*str*)
- **cell_filter** (*str*)

outdir: Path

enabled: bool

parent_source: str

parent_directory: Path | None

include_parent_reference: bool

skip_if_done: bool

resume: bool

count_mode: str

host_species: str

host_oxidation_state: int

vacancy_species: str

vacancy_compensation_charge: int

oxidation_states: dict[str, tuple[int, ...]]

extra_vacancies: int

max_vacancies_cap: int

symprec: float

angle_tolerance: float

mapping_tolerance: float

enumeration_mode: str

max_exact_raw_configs: int

max_exact_unique_configs: int

sample_budget: int

sample_batch_size: int

sample_patience: int

sample_seed: int

sample_max_saved: int

minimum_vacancy_distance: float

backend: str

model: str

task: str

device: str

gpu_id: int

n_workers: int

tf_threads: int

omp_threads: int

chunksize: int

topk_per_vacancy_count: int

energy_normalization: str

optimizer: str

fmax: float

max_steps: int

relax_mode: str

cell_filter: str

dopingflow.vacancies.**parse_oxidation_state_arrays**(*section*)

Parse the required flat parallel oxidation-state arrays.

Parameters
section (*dict[str, Any]*)

Return type
dict[str, tuple[int, ...]]

dopingflow.vacancies.**parse_vacancy_config**(*raw*, *root*)

Parameters

- **raw** (*dict[str, Any]*)
- **root** (*Path*)

Return type
VacancyConfig

dopingflow.vacancies.**reachable_charge_scenarios**(*dopant_counts*, *oxidation_states*, *host_oxidation_state*, ***, *max_examples_per_charge=32*)

Combine compact oxidation-state populations and deduplicate total charges.

Parameters

- **dopant_counts** (*dict[str, int]*)
- **oxidation_states** (*dict[str, Sequence[int]]*)
- **host_oxidation_state** (*int*)
- **max_examples_per_charge** (*int*)

Return type
list[dict[str, *Any*]]

dopingflow.vacancies.**determine_vacancy_counts**(*scenarios*, ***, *compensation_charge*, *extra_vacancies*, *max_vacancies_cap*, *available_sites*)

Parameters

- **scenarios** (*Sequence[dict[str, Any]]*)
- **compensation_charge** (*int*)
- **extra_vacancies** (*int*)
- **max_vacancies_cap** (*int*)
- **available_sites** (*int*)

Return type
tuple[list[int], dict[str, *Any*]]

dopingflow.vacancies.**charge_scenarios_for_count**(*scenarios*, *n_vacancies*, *compensation_charge*)

Parameters

- **scenarios** (*Sequence[dict[str, Any]]*)
- **n_vacancies** (*int*)

- **compensation_charge** (*int*)

Return type
list[dict[str, *Any*]]

dopingflow.vacancies.**periodic_vacancy_distances**(*structure*, *removed_indices*)

Parameters

- **structure** (*Structure*)
- **removed_indices** (*Sequence[int]*)

Return type
tuple[float | None, float | None, float | None]

dopingflow.vacancies.**enumerate_vacancy_orbits**(*symmetry_parent*, *relaxed_parent*, *symmetry_vacancy_indices*, *relaxed_vacancy_indices*, *n_vacancies*, *cfg*)

Enumerate exact or sampled vacancy combinations, reduced by parent symmetry.

Parameters

- **symmetry_parent** (*Structure*)
- **relaxed_parent** (*Structure*)
- **symmetry_vacancy_indices** (*Sequence[int]*)
- **relaxed_vacancy_indices** (*Sequence[int]*)
- **n_vacancies** (*int*)
- **cfg** (VacancyConfig)

Return type
tuple[list[dict[str, *Any*]], dict[str, *Any*]]

dopingflow.vacancies.**vacancy_config_fingerprint**(*cfg*, **source_paths*)

Parameters

- **cfg** (VacancyConfig)
- **source_paths** (*Path*)

Return type
str

dopingflow.vacancies.**discover_selected_parents**(*outdir*)

Parameters
outdir (*Path*)

Return type
list[dict[str, *Path* | str]]

dopingflow.vacancies.**map_parent_sites**(*symmetry_parent*, *relaxed_parent*, ***, *tolerance*, *parent_id*)

Parameters

- **symmetry_parent** (*Structure*)
- **relaxed_parent** (*Structure*)
- **tolerance** (*float*)

- **parent_id** (*str*)

Return type
dict[int, int]

dopingflow.vacancies.**run_vacancies**(*raw*, *root*, *, *config_path=None*)

Parameters

- **raw** (*dict[str, Any]*)
- **root** (*Path*)
- **config_path** (*Path | None*)

Return type
Path

dopingflow.vacancies.**run_vacancies_from_toml**(*config_path*)

Parameters
config_path (*Path*)

Return type
Path

dopingflow.utils.symmetry module

dopingflow.utils.symmetry.**build_sublattice_symmetry_permutations**(*parent*, *sublattice_indices*, *, *symprec*, *angle_tolerance=5.0*)

Return deterministic symmetry permutations for a selected sublattice.

Parameters

- **parent** (*Structure*)
- **sublattice_indices** (*Sequence[int]*)
- **symprec** (*float*)
- **angle_tolerance** (*float*)

Return type
list[*ndarray*]

dopingflow.utils.symmetry.**canonical_occupancy_key**(*labels*, *permutations*)

Canonicalize a sublattice label vector under supplied permutations.

Parameters

- **labels** (*ndarray | Sequence[int]*)
- **permutations** (*Sequence[ndarray]*)

Return type
bytes

dopingflow.utils.parallel module

dopingflow.utils.pymatgen_helpers module

Module contents

===== END =====

PYTHON MODULE INDEX

d

INDEX

E

F

G

H

I

L

M

N

O

P

R

S

T

V

W

X